ABSTRACT

Frequent itemset mining in large, dense databases poses significant challenges, primarily due to the substantial number of 2-itemsets generated, especially when transactions contain long lists of elements. As the number of elements in each transaction increases, the associated computational overhead in terms of memory usage and processing time grows exponentially. Traditional methods typically process all elements in every transaction, resulting in inefficiencies with regard to runtime and resource utilization. To address these limitations, this project work presents a novel algorithm called **"Bitvector based Frequent Pattern using Discarded Elements Prudently – (BFPDEP)"** This approach leverages binary representation to streamline computations, significantly enhancing the efficiency of the mining process. By selectively replacing actual elements with discarded elements in transactions that contain large numbers of items, BFPDEP reduces the computational load associated with both memory usage and runtime, thereby optimizing performance in dense datasets.

TABLE OF THE CONTENT

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iv
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	ABBREVIATIONS	xii
1.	**INTRODUCTION**	1
	1.1 PREFACE	1
	1.2 INTRODUCTION TO DATA MINING	1
	1.3 FREQUENT ITEMSET MINING	2
	1.4 BACKGROUND TO THE PROJECT WORK	3
	1.5 SCOPE OF THE PROJECT WORK	4
	1.6 MOTIVATION	4
	1.7 OBJECTIVES OF THE PROJECT WORK	5
	1.8 PROJECT METHODOLOGY	5
	1.9 PROJECT CYCLE	6
2.	**LITERATURE REVIEW**	8
	2.1 MINING SEQUENCIAL PATTERN	8
	2.1.1 LIMITATIONS OF APRIORI	8
	2.2 PARALLEL ALGORITHMS FOR DISCOVERY OF ASSOCIATION RULES	9
	2.2.1 LIMITATIONS OF ECLAT ALGORITHM	10

	2.3 SECURE ASSOCIATION RULE MINING USING BI-ECLAT ALGORITHM ON VERTICALLY PARTITIONED DATABASES	10
	2.2.1 LIMITATIONS OF BI-ECLAT	11
	2.4 IMPROVING EFFICIENCY	11
3.	**SYSTEM ANALYSIS**	14
	3.1 EXISTING SYSTEM	14
	3.1.1 PROBLEMS RELATED TO CANDIDATE GENERATION	14
	3.1.2 PROBLEMS RELATED TO MEMORY	15
	3.2 PROPOSED SYSTEM	16
4.	**HARDWARE AND SOFTWARE DESCRIPTION**	18
	4.1 HARDWARE REQUIREMENT	18
	4.2 SOFTWARE REQUIREMENT	20
	4.3 ABOUT ECLIPSE	22
	4.4 ABOUT GITHUB	24
	4.5 DATASET USED	26
	4.5.1 REAL DATASETS	26
	4.5.1.1 MUSHROOM	27
	4.5.1.2 CHESS	27
	4.5.1.3 CONNECT-4	27
	4.5.2 SYNTHETIC DATA	28
	4.5.2.1 KEY FEATURES OF IBM	28

4.5.2.2 PARAMETERS FOR DATA GENERATION 29
5. SYSTEM ARCHITECTURE 30
6. MODULE DESCRIPTION 31
6.1 DATA PREPROCESSING MODULE 31
6.1.1 SORT THE DATASET 31
6.2 DISTINCT ELEMENT AND ITEM COUNT GENERATION MODULE 32
6.2.1 TOP-DOWN BOTTOM- UP RETRIEVAL 32
6.2.2 DISTINCT ELEMENT AND ITEM COUNT IDENTIFICATION 34
6.3 DATA SEGMENTATION MODULE 34
6.4 BIT VECTOR TRANSFORMATION MODULE 36
6.5 PRUNING AND OPTIMIZATION MODULE 38
6.6 FREQUENT ITEMSET GENERATION MODULE 39
7. SYSTEM DESIGN 48
7.1 USE CASE DIAGRAM 48
7.1.1 PREPROCESS THE DATA 48
7.1.2 FIND DISTINCT ELEMENTS 49
7.1.3 BISECT THE DATA TO FIND THE DISCARDED 50
7.1.4 BIT VECTOR REPRESENTATION 51

	7.1.5 PROPOSED BFPDEP	52
	7.2 SEQUENCE DIAGRAM PREPROCESSING	53
	7.2.1 DISTINCT ELEMENTS	54
	7.2.2 BIT VECTOR REPRESENTATION	55
8.	**TESTING AND IMPLEMENTATON**	56
	8.1 UNIT TESTING	56
	8.2 TESTING CONDUCTED IN THIS PROJECT	57
	8.3 MOCKING (WITH MOCKITO)	58
	8.4 IMPLEMENTATON	59
9.	**CONCLUSION**	63
10.	**FUTURE ENHANCEMENT**	64
	10.1 FUTURE RECOMMENDATIONS	64
	APPENDIX I	65
	SAMPLE CODING	65
	APPENDIX II	69
	SCREENSHOTS	69
	REFERENCES	72

LIST OF TABLES

TABLE NO	DESCRIPTION	PAGE NO
4.1	MINIMUM AND RECOMMENDED HARDWARE REQUIREMENTS	20
6.1	SAMPLE DATA TO SORT	31
6.2	SORTED INPUT DATABASE	32
6.3	SAMPLE DATABASE TO TEST	34
6.4	REDUCED SAMPLE DATABASE	35
6.5	ACTUAL DATABASE WITH DISCARDED DATABASE REPRESENTED IN BINARY	37
6.6	ACTUAL DATABASE WITH DISCARDED DATABASE REPRESENTED IN BINARY WITH PRUNED DATA	38
6.7	FREQUENT ITEMSETS	47
8.1	COMPARISON RELATED TO CANDIDATE GENERATION	60

LIST OF FIGURES

FIGURE NO	DESCRIPTION	PAGE NO
1.1	THE TAXONOMY OF THE FREQUENT ITEMSET MINING ALGORITHMS	3
1.2	PROJECT CYCLE EMPLOYED IN THE PROJECT WORK	7
3.1	OVERALL ARCHITECTURAL DIAGRAM OF THE PROPOSED BFPDEP ALGORITHM	16
5.1	SYSTEM ARCHITECTURE OF BFPDEP ALGORITHM	30
6.1	BISECTDATA PROCEDURE PROCESS DIAGRAM	33
6.2	REPRESENTING HOW TO FIND THE DISCARDED ELEMENTS	36
7.1	USE CASE DIAGRAM OF THE PREPROCESSING	48
7.2	USE CASE DIAGRAM OF THE FINDING THE DISTINCT ELEMENTS ALONG WITH ITS COUNT	49
7.3	USE CASE DIAGRAM OF THE FINDING THE STORE THE DISCARDED AND ACTUAL DATABASE SEPARATELY	50
7.4	USE CASE DIAGRAM OF THE BINARY REPRESENTATION OF THE DATABASE	51

7.5	USE CASE DIAGRAM OF THE PROPOSED BFPDEP ALGORITHM	52
7.6	SEQUENCE DIAGRAM OF THE PREPROCESSING PROCEDURE	53
7.7	SEQUENCE DIAGRAM TO FIND THE DISTINCT ELEMENTS PROCEDURE	54
7.8	SEQUENCE DIAGRAM TO REPRESENT DATA IN BINARY	55
8.1	GRAPH REPRESENTING THE COMPARISON RELATED TO CANDIDATE GENERATION	60
8.2	GRAPH REPRESENTING THE COMPARISON RELATED TO EXECUTION TIME CONSUMED	61
8.3	GRAPH REPRESENTING THE COMPARISON RELATED TO MEMORY CONSUPTION	61

ABBREVIATIONS

DM – Data Mining

DA – Data Analytics

FIM – Frequent Itemset Mining

MS – Minimum Support

KDD – Knowledge Discovery in Database

TDB – Transaction Database

FPTree – Frequent Pattern Tree

BFS – Breadth First Search

DFS – Depth First Search

MFI – Maximal Frequent Itemset

CFI – Closed Frequent Itemset

DIC - Dynamic Itemset Counting

BFPDEP – Bit vector based Frequent Pattern using Discarded Elements Prudently

TDBU – Top-Down Bottom Up

ADB – Actual Database

DDB – Discarded Database

TID – Transaction ID

EOF – End Of the File

CHAPTER 1

INTRODUCTION

1.1 PREFACE

Nobody on the planet could have predicted when the modern-day magical device named computer was invented in the nineteenth century that it would become an indispensable part of everyone's life today. This modern-day magical device has advanced to the point where it is used in every industrial sector, including academia, research, healthcare, communication, science, data science, AI and many more. Computer-based technologies have developed and expanded to such an extent that they have completely altered the way that commerce and trade are viewed globally. The information and the dynamic data gathered is vital in many fields and helps humans make decisions, do large-scale calculations and computations without errors, and develop artificial general intelligence AGI.

Access to the vast amounts of online data that are accessible via the web and data repositories has been enhanced by developments in computer and information technology. To get the intended results for both individuals and businesses, the majority of these data archives are regrettably not being used and processed correctly. It is an enormous undertaking to manually process the vast amounts of data and the important concern is that the data that is growing sharply and huge junk of data is being stored every day. In order to solve this important limitation, the data must be analyzed and mined in order to find relevant and helpful patterns that serve as a guide to create error-free decisions and rules to benefit the humans.

1.2 INTRODUCTION TO DATA MINING

The study of finding hidden patterns and designs from the raw data is known as data mining. The art of finding hidden information or patterns that is unknown to humans previously is data mining and the foremost aim of data mining is to discover these hidden secret patterns. Recently, the data mining vertical has grown significantly,

attracted a lot of attention from every sector and individuals, and changed extensively and rapidly. Data mining, a compelling new breakthrough with enormous potential to help and support businesses dealing with raw data information and improve their business related to the customer support, financially and expand their horizons. The main goal of knowledge discovery in databases (KDD) is to uncover interesting, dubious, yet practical information in the unprocessed data. This helpful information helps businesses create predict new data patterns, improve decision-making in business, and give graphical data perception to assist people in finding clandestine information and allow unquestionable access to the raw data available across the web.

1.3 FREQUENT ITEMSET MINING

One of the most crucial data mining tasks is finding new patterns from raw databases that have never been found before. Pattern analysis, artificial general intelligence, information-based systems, and machine learning are all components of the complex field of data mining. According to the author Agarwal et.al, the vast amount of raw data that is available in the real world is mined and used efficiently by the data mining. Frequent itemset, patterns and the relationships between different entities are used in different association rule mining techniques like FIM and association rule mining effectively to help the business houses running smoothly.

Numerous research projects are underway to develop an algorithm that is nearly perfect for mining frequent items, and there are numerous efficient and effective algorithms based on frequent item set mining that are easily accessible worldwide. In terms of computational overheads, however, the most crucial query is "Are the algorithms prevailing across the globe is cheap and affordable?" Frequent itemset mining has been increasingly important in recent years for most businesses wanting to improve their customer service and upselling operations. Many academics are encouraged by this FIM to study and analyze other methods and strategies linked to frequent pattern mining that is efficient and has low operational and computational costs.

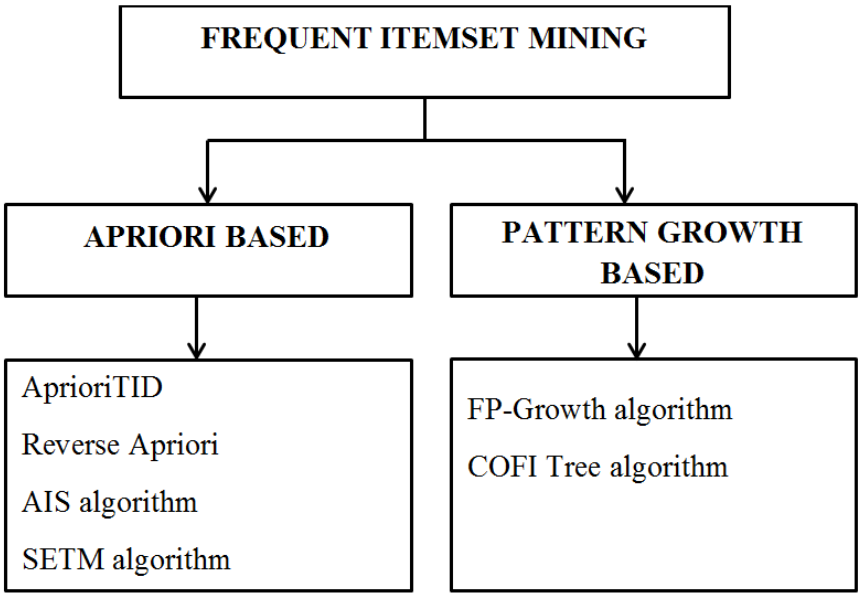


Figure 1.1: The taxonomy of the frequent itemset mining algorithms

1.4 BACKGROUND TO THE PROJECT WORK

Not much progress is made in the Knowledge Data Discovery process to transform unrefined data into meaningful and useful information that people can use. There is often a lag in fully understanding the data in relation to various business needs, such as profit, customer satisfaction, marketing of new products, because the vast amount of data in a database often surpasses the ability to investigate, analyze, and mine it effectively and affordably.

The vertical Frequent Itemset Mining - FIM, has gained significant traction and significance among academics ever since trade and business have advanced to become more worldwide and broken the country's physical barriers. In order to compete with multinational corporations, the business firms today must adapt and change according to the varying needs of the business and according to the latest trends. They must also benefit from handling unknown data and turning it into conceivable, usable and meaningful data. To improve and increase the amount of business exchanges worldwide, numerous association rules based on frequent itemset mining techniques

are produced and proposed by different research experts to help the business houses and other research scholars intend to discover a new algorithm in this ever-evolving field of data mining.

1.5 SCOPE OF THE PROJECT WORK

The research work deals with two new algorithms and methods where the large dataset is divided using top down and bottom-up approach. The subdivided database is then processed to find the frequent itemset using the binary value presentation. Here the support count (Locally found support – Bisect Count) is found for the divided databases and then the support count (Globally found support – Combined Count) is finally calculated after merging the divided database. The unwanted items or the unpromising items are pruned using simple binary calculation on the binary value presentations. This simple approach reduces the time complexities to a large extent, reduces the memory usage considerably and increases the speed of the execution. In order to demonstrate a straightforward method for identifying frequent and maximum frequent itemsets from transactional databases, this research work aims to alleviate the laborious mathematical manipulations and complex computations while pruning the unpromising items found in the state-of-the-art algorithm prevailing in the market presently.

1.6 MOTIVATION

What motivates frequent itemset mining project to be taken so seriously is the key query. Before providing some insight into this investigation, let us first disentangle the historical context of this common itemset mining. This innovative technology was invented two decades back, but it is still evolving and providing new information to the research community to explore and develop new techniques and processes. This is the most astounding factor that motivates and drives any researcher to conduct a regular itemset mining study. Finding the advantages and disadvantages of the current algorithms is necessary before beginning any study. It is also important to determine whether there is a way to improve the current approach.

1.7 OBJECTIVES OF THE PROJECT WORK

The main aim or the objective of this project work is to design and develop a new algorithm named “Bitvector based Frequent Pattern mining using Discarded Elements Prudently- BFPDEP” using top-down bottom-up approach to divide the database and discover FI using simple binary operations to evade the time & memory related complexities. Beside this, the following are the secondary objectives,

1. The planned work must first be developed and designed following a thorough and comprehensive study and survey of the prior works available, including an investigation and comparison of their research to identify any shortcomings or problems.
2. Attributes like candidate generation, execution time consumption and memory usage are focused in this research work. The proposed algorithms aim to reduce the computational overheads.
3. To introduce an efficient pruning mechanism to produce meaningful and useful frequent patterns.
4. To retrieve data from the upper half and the lower half of the large dataset and find the meaningful frequent itemsets with minimum cost and overheads,

1.8 PROJECT METHODOLOGY

This project work proposes the "BFPDEP algorithm," an effective method for mining frequent itemsets. The unique items are first located, and a table is then created to indicate whether or not the item is present by expanding the structures of bit vector data representation. This binary data presentation is used to build a permutation that uses the user-defined support count value to determine whether or not level-wise itemsets are frequent or not. Likewise, the maximum frequent itemset is also found, and the evaluation and experimental results demonstrated that the proposed algorithms can get high levels of efficiency and surpass the other existing methods in terms of memory usage and execution time.

Since the proposed approach makes use of bit vector manipulation and basic addition operations, it is straightforward to implement and the complexity associated with eliminating superfluous items is significantly decreased.

A variety of datasets, including synthetic and real-time ones, are used to test the proposed methodology, which is an ideal method for quickly identifying the frequently occurring itemset. This section highlights the key approaches that were employed in this study.

- **EXAMINING THE PROBLEM**
  After carefully examining the shortcomings of the current algorithms and using the insights gathered from the analysis, the gap between the proposed and current algorithms is closed. This is a critical step in identifying the frequent itemset.

- **FRAMEWORK DESIGN**
  As a tool or software, a practical and reasonably priced framework that addresses the proposed algorithm pseudo code is created. The proposed technique for locating often occurring itemsets in huge datasets uses a number of crucial primary processes and subprocesses that are included in the recently created framework.

- **EVALUATION AND TESTING**
  Using a number of benchmarked criteria, a rigorous evaluation and assessment procedure is conducted to demonstrate the efficacy of the proposed algorithms with respect to various metrics.

1.9 PROJECT CYCLE

The growth of research procedures and approaches used by scholars and practitioners has continuously advanced at a remarkable rate, according to the research community. These developments cover a wide range of methodologies, such as thorough scientific verifications, real-time data analysis, the creation of working

prototypes, frameworks, software programs, tools, and methodical research backed by organized surveys and questionnaires.

The main goal of the suggested research project is to create, develop, and apply novel algorithms as well as supplementary sub-methods that work in unison with the main algorithm. The goal of these sub-methods is to improve the primary algorithm's performance, scalability, and suitability for a range of real-world situations.

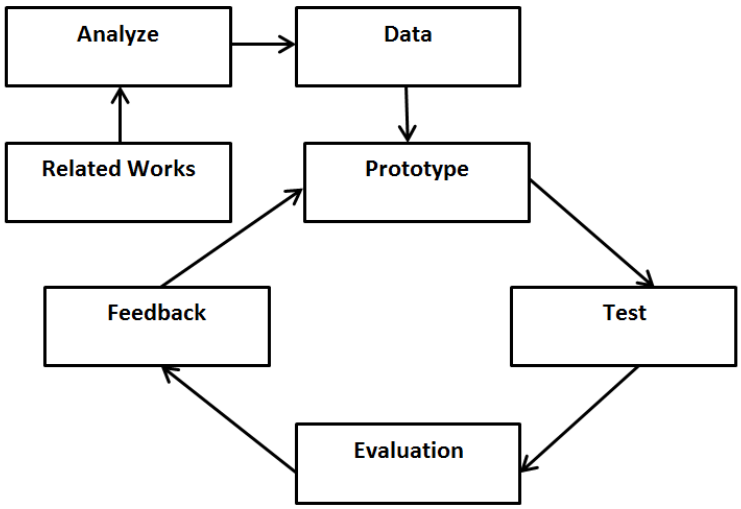


Figure 1.2: Project cycle employed in the project work

To ensure the effectiveness and reliability of the developed methodologies, a comprehensive evaluation process is undertaken. This involves benchmarking the algorithms against established datasets and conducting controlled experiments to assess their accuracy, efficiency, and robustness. The results obtained from these evaluations provide critical insights and validation, confirming the practicality and impact of the proposed solutions.

Consequently, a test-and-evaluate methodology has been adopted as the core research strategy. This approach focuses on iterative testing, performance analysis, and refinement to ensure the outcomes meet academic standards and have practical value. It aims to contribute to the field and support future research and applications.

CHAPTER 2
LITERATURE REVIEW

2.1 TITLE: MINING SEQUENCIAL PATTERN

A common tool for revealing all comparable item arrangements that meet the user-provided minimum support threshold value is the frequent itemset mining method. Agrawal [17] was the first to propose the main method, Apriori, for mining frequent itemsets. The Apriori algorithm is the foundation of many other algorithms, and other writers have developed improved versions of it or techniques based on it. In order to find all frequent 1-itemsets, this algorithm first searches the database. It then goes back and finds all frequent 2-itemsets, 3-itemsets, and so on. After determining the contender's minimum support count, each cycle generates candidate itemsets of length n by merging frequently occurring itemsets of length n – 1. Using a level-wise pruning strategy, the Apriori algorithm finds frequently occurring item sets in transactional databases. This approach iterates to locate the higher itemset (n+1) that adheres to the minimum support count rule after first identifying the often occurring 1-itemset in the transactional database.

2.1.1 LIMITATIONS OF APRIORI

1. Apriori's candidate generation is a laborious procedure that takes a long time and uses a lot of memory to handle large databases.
2. Finding the potential candidates requires many database scans, which significantly strains the algorithm's performance.
3. Compared to the depth-wise approach used by many algorithms, the Apriori's breadth-first technique is significantly slower and takes longer.

Algorithm 1: A-priory algorithm

Data: T - transactions, t - support threshold
Result: all frequent itemsets - F_1,...,F_k

```
Generate frequent 1-itemsets F_1 (support ≥ t);
k := 1 ;
while F_k isn't empty do
    Generate C_{k+1} candidates from F_k ;
    Remove from C_{k+1} itemsets, which have any infrequent subset ;
    Count support for C_{k+1} in T ;
    F_{k+1} := itemsets from C_{k+1} with support ≥ t ;
    k := k+1 ;
end
```

2.2 TITLE: PARALLEL ALGORITHMS FOR DISCOVERY OF ASSOCIATION RULES

To find the potential candidates from the big transactional databases, Zaki's Eclat method employs the Tid-list intersection and vertical data structure. The primary benefit of the Eclat algorithm is that it uses far less memory than the Apriori method since only the tidlists are stored in memory, not the item. By adding the "diffset" approach, which represents just the difference items, the authors Zaki and Gouda [16] improved this algorithm and significantly decreased memory use on dense, huge databases with lengthy transactions.

Data: F_k - frequent patterns of size k, t - support threshold
Result: FP - frequent patterns

```
forall I_i in F_k do
    F_i := { } ;
    forall I_j in F_k, where I_i has the same (k − 1)-prefix and I_j > I_i do
        I_ij := I_i ∪ I_j ;
        tidset(I_ij) := tidset(I_i) ∩ tidset(I_j) ;
        if tidset(I_ij) > t then
            Add I_ij into F_i;
        end
    end
    FP := FP ∪ F_i ;
    Eclat(F_i, s) ;
    Include in FP all itemsets returned by previous step ;
end
```

2.2.1 LIMITATIONS OF ECLAT

1. When transactions are too big to fit in system memory, the Eclat algorithm cannot handle a large database.
2. The Eclat algorithm's join method has several conditional branches, which significantly hinders the algorithm's performance.

The depth-first Eclat method outperforms the majority of current algorithms and is unquestionably faster than the traditional Apriori algorithm.

2.3 TITLE: SECURE ASSOCIATION RULE MINING USING BI-ECLAT ALGORITHM ON VERTICALLY PARTITIONED DATABASES

These are the stages that make up the Bi-Eclat algorithm:

1. Sort the transaction table's frequently occurring items into categories and arrange the tids in decreasing order based on their minimum support values.
2. Create the equivalency class lattice and use the transactional database to produce the candidates, or promising itemsets.
3. To find the often-occurring itemsets, use a pruning technique to eliminate the undesirable alternatives.

Input: atom set S sorted in ascending order of support
Output: frequent itemsets
Procedure candidate_frequent_itemsets( )
For all atoms $A_i \in S$ *do*
  $T_i = \Phi$
  For all atoms $A_j \in S$, *with* $sup(A_j) > sup(A_i)$ *do*
    $R = A_i \cup A_j$;
    $tid\text{-}list(R) = tid\text{-}list(A_i) \cap tid\text{-}list(A_j)$;
    If $|tid\text{-}list(R)| \geq min_sup$
      $S = S \cup \{R\}$; $T_i = T_i \cup \{R\}$;
    End If
  End For
  While $T_i \neq \Phi$ *do candidate_frequent_itemsets* (T_i)
End For

Developed to address the shortcomings of the original Eclat algorithm, the Bi-Eclat algorithm is an enhanced version of the Eclat method.

2.3.1 LIMITATIONS OF BI-ECLAT

1. Due to the tidlist's poor construction and large memory footprint, the Bi-Eclat performs poorly when run on a sparse database.
2. The Bi-Eclat is quite complicated and frequently perplexes the user by using descending order when determining the item frequencies and ascending order when determining the item support count.
3. The Bi-Eclat is heavily reliant on the system's main memory since the full database must fit, and the issue gets worse if a big database is handled.

2.4 IMPROVING EFFICIENCY

Reducing the size of the search region can improve the ability to retrieve frequently occurring itemsets. Various algorithms have been developed for the same purpose. Lessanibahri et al. [5] suggested a method for extracting "frequent itemsets" that fit the given length condition. Prior to building the Frequent Pattern Tree, all itemsets that did not meet the necessary length limit were trimmed. The complete collection of features and rows that are unrelated to datasets is intended to be removed using an efficient pre-processing pruning technique [6].

In order to extract fuzzy association rules, a list structure was created [2]. Here, two pruning strategies were created to reduce the search area and increase the extraction speed of fuzzy association rules.

A number of hash-based techniques have been developed recently to improve the extraction of "frequent items." A method for extracting "frequent itemsets" from a streaming environment was given by Bustio-Martinez et al. [7]. This made use of the ideas of hashing and lexicographic itemset ordering. For mining frequent itemsets, HashEclat, an approximation Eclat algorithm, was suggested [8]. association guidelines. Here, two pruning strategies were created to reduce the search area and increase the extraction speed of fuzzy association rules.

The MinHash technique served as the foundation for this strategy, which took into account the trade-off between mining accuracy and run time. The number of frequently extracted itemsets was constrained by the minimal support criterion. The performance evaluation's F1-measure, time, and error values have been established. To extract frequently occurring itemsets, a method utilizing Locality Sensitive Hashing was introduced [4].

An association rule mining method was presented by Sharmila and Vijayarani [3] using fuzzy logic and whale optimization techniques. The key itemsets and transactions from the dataset were accurately predicted by the suggested method. A co-occurrence-based pruning strategy was proposed to narrow the search space [1]. This employed novel dimensionality reduction techniques and the depth-first search paradigm. The created method increased mining itemsets' efficiency. In addition, the processing speed increased as compared to the previous methods.

An empirical analysis was conducted on the association rules that were taken from "frequent," "closed," and "maximal itemsets." [10]. Data mining tools and statistical interestingness procedures were used to achieve this. This effort ensured a respectably high value of "precision" and "recall" while eliminating a significant number of "redundant rules." A small study was conducted to use statistical measurements, alternative checks, and methods for removing superfluous rules and assessing the attributes' applicability.

For the purpose of locating "non-redundant discriminative itemsets," an algorithm known as "ExCover" was created [9]. Finding patterns with at least one successful transaction is how this is accomplished. For this, the branch and bound pruning method was used. As a consequence, "best-cover patterns" were created. Despite the effectiveness of this approach, a little study on mining itemsets other than closed itemsets has been conducted.

"Non-Derivable itemsets" mining is another way to deal with redundancy. If an itemset's lowest upper bound and highest lower bound values differ, it is considered "non-derivable" [13]. To produce a tight restriction on the support of itemsets, a number of deduction procedures have been proposed. To extract the whole collection

of non-derivable itemsets, an algorithm akin to Apriori was developed [14]. Experiments and in-depth research have demonstrated that this is a condensed and lossless set of objects.

To improve the effectiveness of retrieving such itemsets, Calders and Goethels [15] proposed a Depth-first approach. Here, a method based on the Eclat algorithm was created. It was discovered that these itemsets were strong and succinct. Outlier detection may also be done with such a small dataset [12]. It was discovered that this method outperformed conventional methods based on frequent itemsets, especially in terms of scalability and speed. For the progressive mining of itemsets, a method based on approximate non-derivable itemsets was used [11].

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing systems for frequent itemset mining primarily revolve around classical algorithms like Apriori, Eclat, and their enhanced versions such as Bi-Eclat. The Apriori algorithm, though foundational, suffers from limitations like repeated database scans, high memory consumption, and slower performance due to its breadth-first search strategy. To overcome these drawbacks, the Eclat algorithm introduced a vertical data format and tidlist intersections, significantly improving performance and memory usage. However, it still struggles with large datasets that exceed system memory. To further optimize performance, Bi-Eclat was introduced, incorporating advanced pruning and equivalence class lattices. Although more efficient in dense datasets, Bi-Eclat faces complexity issues and performs poorly on sparse databases. Various recent approaches have aimed to improve these systems by reducing the search space using pruning techniques, fuzzy logic, hash-based structures, and approximation algorithms like HashEclat and ExCover. Despite their advancements, these systems often struggle with redundancy, scalability, and memory limitations when handling large-scale or sparse transactional data.

3.1.1 PROBLEMS RELATED TO CANDIDATE GENERATION

A large number of candidates that are generated during the execution will affect how the computation is carried out since it will take longer. The creation of candidates in large quantities for denser datasets will be a huge hindrance to execution time and slow down the entire process.

For the researchers, memory footprints and search space will be major problems since the large number of candidates created will result in and need additional execution rounds. This has the "out of the memory" issue and necessitates a large amount of computer memory.

It is a criminal waste to produce and advance with candidates that lack promise since many of the candidates generated throughout the execution will be unpromising in terms of supporting factors.

The process will produce a large number of candidates, slow down performance, and use more machine memory if it uses a poor pruning approach.

3.1.2 PROBLEMS RELATED TO MEMORY

One of the primary challenges in frequent itemset mining is efficiently managing memory usage to prevent performance bottlenecks. In large, dense datasets, where there are numerous candidate itemsets with minimal overlap, the memory required to store these itemsets may exceed system capacity, causing excessive memory thrashing and significantly slowing down the mining process. To address this, we must adopt a trade-off strategy that optimizes both memory consumption and the generation of candidate itemsets. This involves reducing the number of candidates while ensuring the memory footprint remains within manageable limits. Compared to current techniques, the goal is to develop a more efficient approach that maintains high performance, even when dealing with large-scale datasets, by minimizing resource overhead and computational complexity.

The purpose of the experimental approach is to determine whether or not an item set should be included in the candidate set. However, in the early phases of candidate creation, when the number of candidates is certain to surpass the number of all possible item combinations, this experimental forecast frequently overestimates. The majority of computers' computing power and accessible memory space can be easily consumed by such heuristics.

The majority of algorithms are based on the Apriori algorithm, which was the foundation for most of them. The Apriori property, also known as the anti-monotonic or downward closure property, states that if an itemset does not adhere to the minimum support consideration, then all of its supersets must also be infrequent. It states that "All the subsets of frequent itemset should be frequent and all supersets of infrequent itemset should be infrequent." The algorithms' use of this downward closure feature

forces them to compute the support count frequently, which adds significantly to the algorithm's time costs because it takes a long time to check the count repeatedly.

3.2 PROPOSED SYSTEM

Frequent itemset mining in large, dense databases presents significant challenges due to the high number of 2-itemsets generated, especially in transactions with long lists of elements. The computational overhead associated with memory usage and processing time increases exponentially as the number of elements per transaction grows. Traditional approaches process all elements in each transaction, leading to inefficiencies in runtime and resource consumption. To overcome these snags, this project work introduces a new algorithm named “Bitvector based Frequent Pattern mining using Discarded Elements Prudently-BFPDEP)” which uses binary representation to ease the computations.

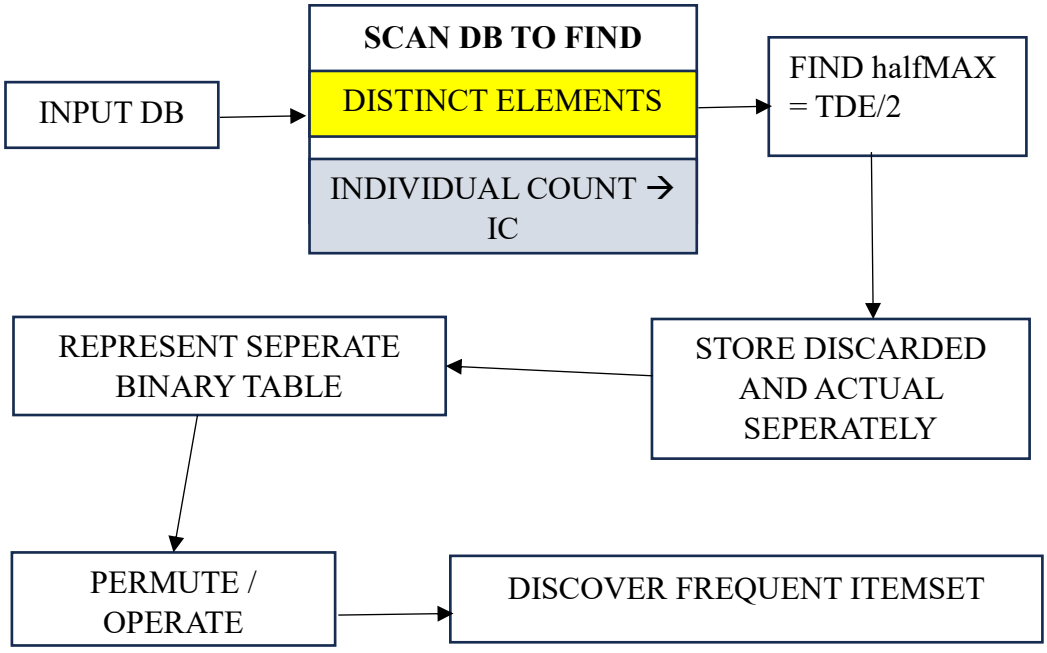


Figure 3.1: Overall architectural diagram of the proposed BFPDEP algorithm

The initial step in the proposed methodology involves identifying the distinct elements present across the dataset. This step not only extracts all unique items but also calculates the frequency of elements within each row, thereby providing valuable insight into the composition and density of data instances. By determining the occurrence of each element per transaction (row), this process sets the foundation for subsequent data segmentation.

Once the distinct elements have been identified, the dataset is strategically partitioned into two components: the Actual Database (ADB) and the Discarded Database (DDB). This classification is based on a straightforward yet effective criterion. Specifically, for each transaction, if the number of elements it contains exceeds half the total number of distinct elements (i.e., total distinct elements ÷ 2), then that transaction is considered overly dense and its elements are transferred to the Discarded Database. In such cases, the original transaction in the ADB is effectively emptied or reduced, while its contents are mirrored in the DDB. This segmentation is key to reducing redundancy and improving the efficiency of subsequent mining processes.

Following this segmentation, both the ADB and DDB undergo a transformation into binary vector representations, where the presence or absence of each element is encoded in binary form. These binary representations serve as the basis for logical operations tailored to each segment. For the Actual Database (ADB), a logical AND operation is applied across the binary vectors. This helps identify frequent itemsets that are consistently present across multiple transactions. Conversely, for the Discarded Database (DDB), a logical OR operation is utilized, allowing the identification of items that appear frequently even though they were initially deemed less significant due to high transaction density.

This two-pronged approach of mining frequent patterns—using AND for ADB and OR for DDB—forms the core of the BFPDEP (Bit vector-based Frequent Pattern mining using Discarded Elements Prudently) algorithm. This method not only enhances pattern discovery efficiency but also maximizes the use of information that might otherwise be overlooked.

CHAPTER 4
HARDWARE AND SOFTWARE DESCRIPTION

4.1 HARDWARE REQUIREMENT

The hardware requirements for a data science project largely depend on the size of the dataset, the complexity of the algorithms, and the type of tasks (e.g., data preprocessing, training machine learning models, visualization). Below is a general guide on the hardware requirements for typical data science projects:

PROCESSOR (CPU)

Type: Multi-core processors (e.g., Intel i7/i9, AMD Ryzen 7/9)

Recommendation: 4 to 8 cores with a high clock speed (3.0 GHz or higher).

Purpose: Handles tasks such as data cleaning, feature engineering, and running basic algorithms.

MEMORY (RAM)

Minimum: 16 GB RAM

Recommended: 32 GB or more, especially for large datasets or deep learning tasks.

Purpose: Sufficient memory is critical to handle in-memory data processing and avoid bottlenecks when working with large datasets.

GRAPHICS PROCESSING UNIT (GPU

Optional but recommended for deep learning or large-scale machine learning tasks.

Type**:** NVIDIA GPUs with CUDA support (e.g., NVIDIA RTX 3060/3090, A100, or V100 for advanced tasks).

Purpose: Accelerates deep learning model training, especially for tasks involving neural networks and large datasets.

STORAGE

Type: Solid State Drive (SSD)

Minimum: 512 GB SSD

Recommended: 1 TB SSD or more, with additional HDD (2 TB) for data storage.

Purpose: Faster read/write operations for loading datasets, and storing large volumes of data and intermediate results.

NETWORK

Internet Speed: Fast internet connection for downloading large datasets, cloud integration, or using external libraries.

Optional: 10 Gbps Ethernet or faster if working on distributed computing or accessing cloud resources frequently.

EXTERNAL DEVICES/PERIPHERALS

Monitors: At least two monitors for better multitasking and coding.

Optional: Cloud access to GPUs or TPUs (such as AWS EC2, Google Cloud, or Azure) for handling massive datasets or resource-intensive tasks.

CLOUD COMPUTING RESOURCES (OPTIONAL)

When local hardware is insufficient, cloud services such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP) offer scalable resources (e.g., GPUs, TPUs, large memory machines) to run complex data science tasks.

SUMMARY OF MINIMUM REQUIREMENTS

Component	Minimum Requirement	Recommended Requirement
CPU	Intel i5, 4 cores	Intel i7/i9, 6-8 cores
RAM	16 GB	32 GB or higher
GPU (optional)	NVIDIA GTX 1060	NVIDIA RTX 3080/3090
Storage	512 GB SSD	1 TB SSD + 2 TB HDD
Network	Stable broadband internet	High-speed internet (1 Gbps)
External Storage	N/A	Cloud storage (AWS, GCP)

Table 4.1: Minimum and recommended hardware requirements

4.2 SOFTWARE REQUEIREMENTS

For he proposed BFPDEP project using Java language, you would need the following software requirements,

JAVA DEVELOPMENT KIT (JDK)

JDK 8 or later: Ensure that the Java Development Kit (JDK) is installed. JDK the necessary tools to develop and run Java applications.

INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

Eclipse IDE: Popular for Java development, supports many plugins for data science and machine learning.

IntelliJ IDEA: A powerful IDE with advanced code assistance, refactoring, and debugging.

NetBeans: Another popular IDE for Java development.

JAVA LIBRARIES FOR DATA SCIENCE

Apache Commons Math: A popular Java library for mathematics and statistics, which provides features like linear algebra, statistics, and random data generation.

ND4J (N-Dimensional Arrays for Java): An efficient scientific computing library that supports multi-dimensional arrays and is integrated with deep learning libraries like DL4J.

Apache Spark with Java API: A powerful distributed computing framework that has a Java API for handling big data and machine learning workflows.

Deeplearning4j: A deep learning library for Java that supports neural networks and can be integrated with Spark.

DATABASE

UCI repository – You can get lots of benchmarked dataset that cam be used to test the prototype and gauge the overall performance of the project

IBM Quest - Used to generate synthetic datasets

VERSION CONTROL

Git: To manage and collaborate on your code repository.

DATA VISUALIZATION TOOLS

JavaFX: Can be used for building graphical user interfaces and simple data visualizations.

JFreeChart: A popular library for creating charts and plots in Java.

BUILD TOOLS

Maven or Gradle: Dependency management and build automation tools for handling Java projects.

TESTING FRAMEWORK

JUnit: For unit testing of Java code.

DATA PROCESSING TOOLS

Hadoop: For distributed storage and processing of large datasets.

Kafka: For real-time streaming and handling of data pipelines.

OTHER TOOLS

Apache POI: For reading/writing Excel files in Java, useful for data manipulation.

Google Guava: An open-source set of common libraries for Java, with useful utilities for working with collections, caching, and more.

4.3 ABOUT ECLIPSE

Eclipse is a popular, open-source Integrated Development Environment (IDE) primarily used for Java development but also supports a wide range of other programming languages such as C, C++, Python, PHP, and more, through plugins. It is widely adopted in the software development community because of its extensibility, powerful features, and large plugin ecosystem. Eclipse is maintained by the Eclipse Foundation.

KEY FEATURES OF ECLIPSE

JAVA DEVELOPMENT TOOLS (JDT)

Eclipse is most commonly used for Java development, and it provides rich support for Java through the Java Development Tools (JDT) plugin. The JDT offers features like code completion, syntax highlighting, error detection, refactoring, and code navigation.

CROSS-PLATFORM

Eclipse is available on multiple platforms, including Windows, macOS, and Linux, making it accessible to developers regardless of their preferred operating system.

PLUGIN-BASED ARCHITECTURE

One of the key features of Eclipse is its modular, plugin-based architecture. This means that new features, languages, and tools can be added to the IDE through plugins, giving Eclipse flexibility to support a wide range of development environments (e.g., for C/C++ development, Android development, web development). The Eclipse Marketplace allows users to discover and install plugins directly from the IDE.

CODE EDITOR

The Eclipse editor provides advanced features such as syntax highlighting, code completion (intellisense), code formatting, and on-the-fly error detection and correction. It supports refactoring, which makes code easier to modify and maintain.

INTEGRATED DEBUGGER

Eclipse includes a powerful built-in debugger that helps developers find and fix bugs. The debugger offers features such as breakpoints, step-by-step execution, variable inspection, and monitoring of program execution.

PROJECT MANAGEMENT

Eclipse provides excellent tools for managing projects, including support for Maven, Gradle, and Ant for building Java projects. It can manage complex, multi-module projects efficiently, helping developers organize code into logical groups.

VERSION CONTROL INTEGRATION

Eclipse has built-in support for version control systems like Git, Subversion (SVN), and CVS through plugins. Developers can clone repositories, manage branches, and commit changes directly from the IDE.

USER INTERFACE

Eclipse's interface is highly customizable, with multiple perspectives tailored for different tasks, such as Java development, debugging, testing, and database exploration. Users can configure views and tools to match their workflow.

TESTING TOOLS

Eclipse integrates with testing frameworks like JUnit for unit testing in Java. Developers can write, run, and debug tests within the IDE, with detailed test result reporting

4.4 ABOUT GITHUB

The GitHub is a web-based platform that provides version control and collaborative software development services, primarily using Git. It is a widely used platform for developers to host, manage, and collaborate on projects, offering powerful tools for versioning, issue tracking, and team collaboration. GitHub plays a central role in the open-source community, enabling developers to share and contribute to projects globally.

KEY FEATURES OF GITHUB

GIT VERSION CONTROL

GitHub is built around Git, a distributed version control system created by Linus Torvalds. Git enables developers to track changes in their code, collaborate with others, and manage multiple versions of a project. With Git, developers can commit changes, create branches, merge updates, and revert to earlier versions of the codebase.

REPOSITORIES (REPOS)

GitHub organizes projects into repositories (often referred to as repos), which store all the files and the history of changes for a project. A repository can be either public (accessible to everyone) or private (restricted access). Repositories are where developers collaborate, review code, and maintain project versions.

BRANCHES

Branches are a core feature of Git that allow developers to work on different features, bug fixes, or experiments independently of the main codebase (often called the "main" or "master" branch). When the feature or fix is ready, it can be merged back into the main codebase.

Developers often create a new branch for each feature or issue they are working on, keeping the main codebase stable.

PULL REQUESTS (PRS)

Pull Requests are used to propose changes from one branch to another. When a developer finishes a feature or fixes a bug, they create a pull request to merge their changes into the main branch.

Pull requests allow team members to review, comment, and discuss the proposed changes before merging them. They are critical in maintaining code quality and fostering collaboration.

FORKING

GitHub allows users to fork (or copy) a repository from another user to their own GitHub account. Forking is commonly used in open-source projects to make changes or improvements to a project without affecting the original repository. Once the changes are ready, users can submit a pull request to the original repository to propose incorporating the changes.

GITHUB ISSUES

GitHub has an integrated issue tracking system, allowing developers to report bugs, suggest new features, or document tasks. Each issue can be assigned to specific developers, labelled, and discussed. Issues help manage and prioritize tasks within a project.

GITHUB ACTIONS

GitHub Actions is a powerful automation feature that allows developers to define custom workflows directly within the repository. These workflows can be triggered by specific events like code pushes, pull requests, or issue creation.

Actions are commonly used for continuous integration (CI) and continuous deployment (CD) pipelines, automating testing, building, and deployment of applications.

GITHUB PAGES

GitHub Pages is a feature that allows developers to host static websites directly from a repository. It's often used for documentation, project portfolios, or blogs.

Developers can configure a repository to serve content using custom domain names and themes.

COLLABORATION AND CODE REVIEW

GitHub facilitates team collaboration through pull requests, inline comments on code, and the ability to review and suggest changes to the code. Teams can use these tools to maintain high code quality and ensure that contributions are thoroughly reviewed before being integrated.

RELEASES

GitHub supports creating releases, which are snapshots of the code at a specific point, typically associated with a new version or milestone in the project. Releases allow developers to package a stable version of the software, providing release notes, binary files, and links to the source code.

COLLABORATION TOOLS

GitHub integrates with many third-party tools and services, including continuous integration (CI) services like Jenkins and CircleCI, issue tracking tools like JIRA, and communication tools like Slack and Discord. This helps streamline the workflow and enhances team collaboration.

4.5 DATASET USED

The dataset used in this project is a transactional dataset, where each record represents a transaction containing a set of items. It is designed to reflect real-world scenarios such as market basket data, where the aim is to find patterns and associations among frequently occurring items. The dataset provides the necessary input for preprocessing and applying the BFPDEP algorithm for efficient frequent itemset mining.

4.5.1 REAL DATASETS

The UCI Machine Learning Repository is a well-known source for datasets used in a variety of machine learning tasks, including frequent itemset mining. Here are some

datasets from the UCI repository that are commonly used for frequent itemset mining, association rule mining, or market basket analysis:

4.5.1.1 MUSHROOM

The Mushroom Dataset from the UCI Machine Learning Repository contains records of different species of mushrooms, with information about whether each species is edible or poisonous. It is often used for classification tasks but can also be adapted for association rule mining or frequent itemset mining by treating the mushroom attributes as items.

- **Size**: 8124 samples
- **Features**: Attributes such as cap shape, odour, gill size, and color, among others.
- **Task**: While it is primarily used for classification, it can also be adapted for frequent itemset mining by treating attributes and values as items.
- **Use Case**: The dataset can be used to predict whether a mushroom is edible or poisonous based on its characteristics, or for discovering patterns of characteristics that frequently co-occur in edible or poisonous mushrooms.

4.5.1.2 CHESS

The Chess (King-Rook vs. King-Pawn) Dataset from the UCI Machine Learning Repository is a specialized dataset related to chess endgames, specifically focusing on situations where one player has a king and a rook, and the other player has a king and a pawn. The dataset is commonly used for classification, but it can also be adapted for pattern mining and other tasks.

- **Size**: 3,196 instances
- **Features**: Categorical features representing chess moves and positions.
- **Task**: Though primarily for classification, can be used for frequent itemset mining by analyzing positional patterns.

4.5.1.3 CONNECT-4

The Connect-4 dataset from the UCI Machine Learning Repository contains data on possible game states from the Connect-4 game, a two-player connection game

where players take turns dropping colored discs into a vertically suspended grid. The goal of the game is to connect four of one's own discs in a row, column, or diagonal.

- **Description**: Contains possible game states from the Connect-4 game.
- **Features**: Categorical data representing the game board configuration.
- **Task**: Can be adapted for pattern mining by analyzing the board configurations and move patterns.
- **Size:** 67,557 instances (rows)
- **Attributes:** 42 categorical attributes (each representing a position on the 6x7 Connect-4 grid)
- **Task:** Classification or pattern mining (e.g., frequent itemset mining)
- **Use Case:** Can be used for predicting game outcomes or analyzing game board configurations for patterns.

The dataset is typically available in a CSV format, where each row corresponds to a game board configuration, and each column corresponds to a specific position on the grid. The class label (game outcome) is present in the last column.

These datasets are well-suited for frequent itemset mining, allowing you to explore patterns and associations between items or categorical features in a variety of contexts like retail transactions, product recommendations, or customer behaviour.

4.5.2 SYNTHETIC DATA

The IBM Quest Synthetic Data Generator is a well-known tool for generating synthetic datasets, particularly for benchmarking and evaluating the performance of frequent itemset mining, association rule mining, and other data mining algorithms. The Quest data generator was designed by IBM's Almaden Research Centre and has been used extensively in research papers and experiments related to pattern mining, especially in the context of market basket analysis.

4.5.2.1 KEY FEATURES OF IBM

Synthetic Data Creation: It generates synthetic transaction data based on user-specified parameters like the number of transactions, the number of items, the average transaction length, and the number of frequent patterns.

Market Basket Model: The generated data simulates typical market basket data, where each transaction consists of a subset of items, and the items are drawn from a larger pool. These items often exhibit co-occurrence, making the data suitable for mining association rules and frequent itemsets.

Customizable Parameters: The tool allows users to specify various parameters to control the size and complexity of the data. This makes it easy to create datasets with specific properties for testing the scalability and performance of different algorithms.

Scalability: It can generate datasets of different sizes, from small ones (a few thousand transactions) to very large ones (millions of transactions) for scalability testing.

4.5.2.2 PARAMETERS FOR DATA GENERATION

The Quest Synthetic Data Generator allows you to control several important parameters to simulate different kinds of transactional data:

1. Number of Transactions (T): The total number of transactions (e.g., purchases) in the dataset.
2. Number of Items (I): The total number of unique items in the dataset. Items can be products or entities involved in a transaction.
3. Average Transaction Length (L): The average number of items per transaction.
4. Number of Frequent Itemsets (FI): The number of frequent itemsets to be embedded in the dataset. Frequent itemsets are subsets of items that co-occur in transactions with a specified minimum support.
5. Average Length of Frequent Itemsets (FL**)**: The average length of frequent itemsets to be embedded in the data.
6. Support Threshold: Defines the minimum support for frequent itemsets.

CHAPTER 5
SYSTEM ARCHITECTURE

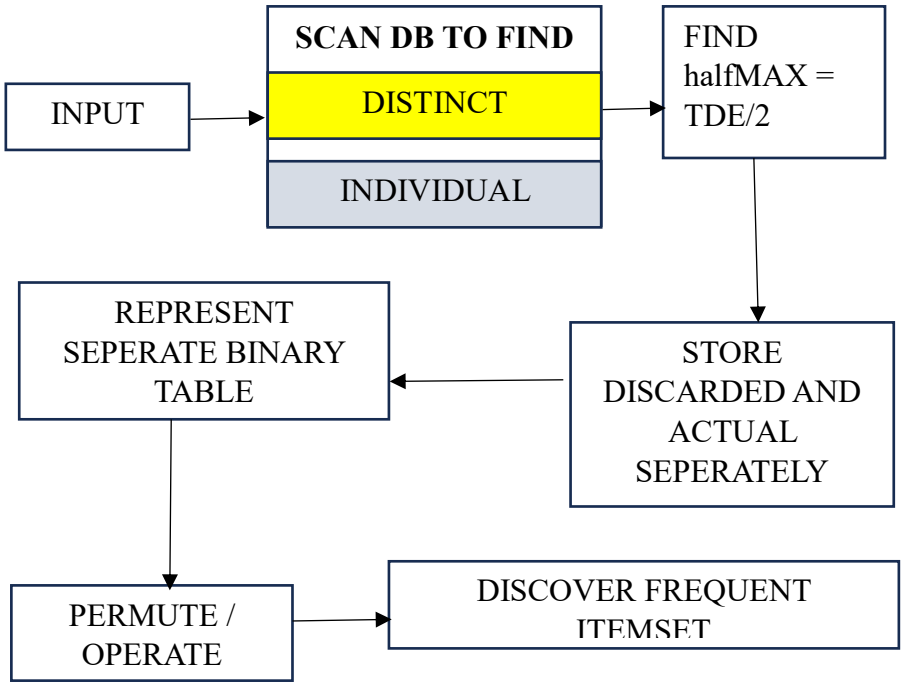


Figure 5.1 System architecture of BFPDEP Algorithm

The proposed BFPDEP algorithm comprises of many procedures and they are illustrated here in this section. The first procedure is to find the distinct elements which also finds the number of elements in each row with its count.

After finding the distinct elements, the database is separated into actual ADB and discarded database DDB using a simple calculation. If the total number of elements present in each row is greater than the value of total number of the distinct elements divided by 2, the entire elements are replaced by the discarded elements. The binary representation of the actual DB and discarded DB are used to perform separate binary operations (AND for ADB, OR for DDB) to unearth the frequent itemsets.

CHAPTER 6
MODULE DESCRIPTION

6.1 DATA PREPROCESSING MODULE

The preprocessing module in BFPDEP converts the raw transactional data into a structured and uniform format. It organizes the raw transactional data by sorting items within each transaction and removing any repetitions. This step prepares the data for further processing by ensuring it's clean, consistent, and ready for frequency analysis.

6.1.1 SORT THE DATASET

In order to guarantee that the input data is sorted before being fed into the main process, this approach significantly cuts down on execution time by using a straightforward sorting algorithm to order the rows.

TID	Transactions
1	I, L, J, M
2	J, M, K
3	J, I, L, M
4	I, J, M, K
5	K, M, I, J, L
6	K, J, L

Table 6.1: Sample data to sort

The table 6.1 shows that there are six rows in the sample database {1, 2, 3, 4, 5, 6} and five items {I, J, K, L, M}.

The process for sorting the transactions process sorts the data one transactional row at a time and uses a straightforward way to finish the
sorting process. One by one, the input items are retrieved and checked to see if they are greater than the next item in the transaction. If not, they are compared to the following item, and if this is the case, they are switched and rearranged as in the worked-out example in the next section.

PROCEDURE SortITEM (Transactional Row T)

Input: Transactional Row T

Output: Reordered row

```
(1) Count total Items  in the row T and store in Rcount
(2) ∀  Item I ∈ Rcount Do begin
(3) ∀ Item I+1 ∈ Rcount Do begin
(4) Check IF ( Item[I] >=Item[ I+1] )
    Swap ( Item[I] , Item[I+1] )
(5) End For
(6) End For
(7) RETURN Reordered row
(8) End Procedure
```

Every transactional row is retrieved from sample table 6.1 and iterated from the array's zero position to its upper bound value. This iteration is then nested with another one that begins at a location that has been increased by one to the array's upper bound value. Sorting is now made possible by comparing the items that were retrieved from the relevant iterations. Should the retrieved item meet the specified condition, the relevant values will be switched. The user will receive the sorted items back after the iteration loops are finished

TID	Transactions
1	I, J, L, M
2	J, K, M
3	I, J, L, M
4	I, J, K, M
5	I, J, K, L, M
6	J, K, L

Table 6.2: Sorted input database

6.2 DISTINCT ELEMENT AND ITEM COUNT IDENTIFICATION MODULE

6.2.1 TOP-DOWN BOTTOM-UP RETRIEVAL

The proposed approach optimizes data retrieval by implementing a bisecting mechanism that drastically minimizes the number of database scans or iterations.

Rather than scanning the entire database, it begins by checking if the input file has not yet reached the end (EOF). It then retrieves the first row while simultaneously accessing the EOF row.

Next, a comparison is made between the upper bound of the data and the value of EOF minus one. If this condition is met, it indicates that the midpoint of the dataset has been reached. At this point, the loop terminates, ensuring that only half of the database is scanned.

This technique effectively reduces processing time and computational resources by leveraging the bisection process. Instead of performing repetitive scans across the entire database, the approach narrows down the data to the necessary portion, halting once the middle of the dataset is reached. By cutting the number of iterations in half, it significantly enhances efficiency, making it particularly useful for large datasets where frequent scanning is resource-intensive. In summary, this mechanism focuses on optimizing retrieval by reducing full database scans, leveraging comparisons between the initial row and EOF, and terminating early when the midpoint is reached. The process diagram is shown in the figure 6.1.

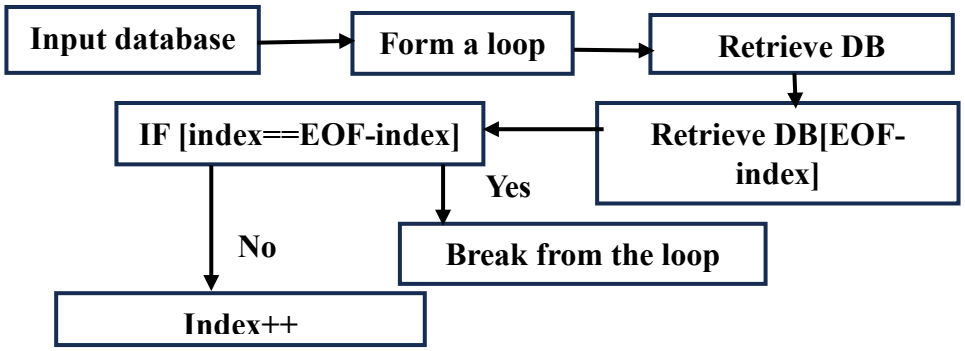


Figure 6.1: BisectData procedure process diagram

EXPLANATION

1. **Initialization**: The function initializes two pointers: start (first row) and end (EOF, last row).
2. **Empty Check**: It checks if the database is empty and returns a message if it is.
3. **Bisecting Process**: The loop runs while the start pointer is less than the end pointer.

4. **Midpoint Calculation**: At each iteration, it calculates the midpoint of the dataset and compares the values at the start and end pointers to see if the midpoint is reached.

5. **Pointer Adjustment**: Depending on the comparison, the algorithm adjusts the start or end pointer, reducing the search space.

6. **Termination**: The loop terminates once the midpoint is reached or the search is narrowed down to a small portion of the data, effectively cutting the number of scans in half.

6.2.2 DISTINCT ELEMENT AND ITEM COUNT IDENTIFICATION

TID	ELEMENTS	Item Count
1	A, B, D, E, H	5
2	B, C, E, K	4
3	A, B, D, E, H, J, K, L, M	9
4	A, B, C, E, G, J, K	7
5	A, B, C, D, E, I, L	7
6	B, C, D	3
7	A, C, D, E, F, H, J, K	8
	Total Elements	**43**

Table 6.3: Sample database to test

Considering the database shown in table 6.3. There are thirteen different elements, *E* = {A=5, B=6, C=5, D=5, E=6, F=1, G=1, H=3, I=1, J=3, K=4, L=2, M=1} and seven transactions TID = {1; 2; 3; 4; 5; 6; 7}.

The sample database shown in the table 1 is considered for the experimental evaluation. The findDISTINCT procedure finds the distinct elements DE= {A, B, C, D, E, F, G, H, I, J, K, L, M}. The count of the distinct elements is found to be 13 which is stored in TDE. The half-MAX is computed using the TDE value, which is 6.

6.3 DATA SEGMENTATION MODULE

The first row T1 = {A, B, D, E, H} and its *rowLength* RL is 5 which is less than the half-MAX so the actual elements are represented in the *RES* along with the

count 5. The second row T2 = {B, C, E, K} and its *rowLength* RL is 4 which is less than the half-MAX so the actual elements are represented in the *RES* along with the count 4. The third row T3= {A, B, D, E, H, J, K, L, M} and its *rowLength* RL is 9 which is greater than the half-MAX so the discarded elements {C, F, G, I} using the set difference is found and represented in the *disRES* along with the count 4 as shown in the table 6.4.

Procedure PrudentDB (Input database DB)

INPUT: Database DB
OUTPUT: Restructured DB

1. *Scan the DB*
2. *Initialize RowCount = 0*
3. *UE [][]=empty*
4. *disRES [][]=Empty*
5. *RES [][]=e\Empty*
6. *While [Row ≠ EOF] do*
7. *Find the Rowlength →RL*
8. *DistinctElement DE = findDISTINCT(Row)*
9. *Find the total length of DE → TDE*
10. *Half-MAX = TDE / 2*
11. *If [RL > half-MAX] do*
12. *disRES = Row \ DE*
13. *Else*
14. *RES = Row*
15. *End If*
16. *End While*

Return RES, disRES
End procedure

ACTUAL ELEMENTS		
TID	**ELEMENTS**	**Item count**
1	A, B, D, E, H	5
2	B, C, E, K	4
3	B, C, D	3
DISCARDED ELEMENT		
TID	**ELEMENTS**	**Item count**
1	C, F, G, I	4
2	D, F, H, I, L, M	6
3	F, G, H, J, K, M	6
4	B, G, I, L, M	5
Total Elements		**33**

Table 6.4: Reduced sample database

From the table 6.4, it is quite clear that the sample database size is reduced considerably from 43 total elements to 33 elements during the initial process which in turn reduces the 2-itemsets candidates generated. This reduction in turn will decrease the execution time, search space and memory consumption considerably. For a sample data with 7 rows the percentage of reduction in the size of the database is around 15% and when we deal with very large dense databases, the reduction in the size will be more beneficial and useful in the reduction of the computational overheads.



Figure 6.2: Representing how to find the discarded elements

The above figure is a self-explanatory picture to demonstrate the working of the discarded elements depending on the element count in each row present in the database.

6.4 BIT VECTOR TRANSFORMATION MODULE

The fundamental concept is to transform the elements shown in the table 2 into bit vectors and then use basic computations like AND/OR operations to identify the frequent itemsets. An element is tagged with a "1" if it is present and with a "0" otherwise. Table 6.5 displays the binary table.

PROCEDURE BinaryCONV(Database RD, Distinct DE)

INPUT: Database RD, Distinct DE
OUTPUT: Binary
BEGIN:

1. Load the reduced database RD
2. Fetch each Row R in RD
3. For each Element E in R do
4. IF [DE present in E] then
5. Mark as "1" in Binary table
6. Else
7. Mark as "0" in Binary table
8. End IF
9. End FOR

RETURN Binary table
END PROCEDURE

In the table 6.5, "*In actual database ADB '1' represents an element's presence and '0' represents its absence whereas in discarded database DDB '0' represents an element's presence and '1' represents its absence.*"

ACTUAL DB					**DISCARDED DB**				
TID	**1**	**2**	**3**		**TID**	**1**	**2**	**3**	**4**
A	1	0	0		**A**	0	0	0	0
B	1	1	1		**B**	0	0	0	1
C	0	1	1		**C**	1	0	0	0
D	1	0	1		**D**	0	1	0	0
E	1	1	0		**E**	0	0	0	0
F	0	0	0		**F**	1	1	1	0
G	0	0	0		**G**	1	0	1	1
H	1	0	0		**H**	0	1	1	0
I	0	0	0		**I**	1	1	0	1
J	0	0	0		**J**	0	0	1	0
K	0	1	0		**K**	0	0	1	0
L	0	0	0		**L**	0	1	0	1
M	0	0	0		**M**	0	1	1	1

Table 6.5: Actual database with discarded database represented in binary

6.5 PRUNING AND OPTIMIZATION MODULE

In the initial pruning step, the elements which does not meet the minimum support threshold value i.e. the element count is less than the given minimum support value is pruned. Here the Elements F, G, I, L, M are pruned.

Minimum Support value=3

Element: { No of ones in Actual DB } + { No of zeros in Discarded DB }

F: 0+1 = 1 < 3

G: 0+1 = 1 < 3

I: 0+1 = 1 < 3

L: 0+2 = 2 < 3

M: 0+1 = 1 < 3

ACTUAL DB					DISCARDED DB				
TID	**1**	**2**	**3**		**TID**	**1**	**2**	**3**	**4**
A	1	0	0		**A**	0	0	0	0
B	1	1	1		**B**	0	0	0	1
C	0	1	1		**C**	1	0	0	0
D	1	0	1		**D**	0	1	0	0
E	1	1	0		**E**	0	0	0	0
F	*0*	*0*	*0*		*F*	*1*	*1*	*1*	*0*
G	*0*	*0*	*0*		*G*	*1*	*0*	*1*	*1*
H	1	0	0		**H**	0	1	1	0
I	*0*	*0*	*0*		*I*	*1*	*1*	*0*	*1*
J	0	0	0		**J**	0	0	1	0
K	0	1	0		**K**	0	0	1	0
L	*0*	*0*	*0*		*L*	*0*	*1*	*0*	*1*
M	*0*	*0*	*0*		*M*	*0*	*1*	*1*	*1*

Table 6.6: Actual database with discarded database represented in binary with pruned data

6.6 FREQUENT ITEMSET GENERATION MODULE

The probable candidates which are constructed from the distinct elements is showcased below and the procedure *ProbableCandidates* employs a recursive technique to execute the same code repeatedly until the generated candidate length equals to the number of distinct elements produced.

PROCEDURE ProbableCandidates (DistinctItem D)

Output: Candidates without repetitions

1. Find the total number of distinct items →Count
2. Initialize Flag = 1, Temp=ϕ, Result=ϕ
3. While [Flag <= Count] do begin
    a. IF (Flag = 1) do
        i. Add Temp=D [Flag] ∪ D [Flag+1]
        ii. STORE Result ← Temp
    b. ELSE
        i. Add Result = Temp ∪ D [Flag]
        ii. STORE Result
    c. END IF
    d. Increment Flag by 1
4. END WHILE
5. RETURN Result

Let us consider that the user defined minimum support is 3, then five elements shown in the table 6.6 with different backcolor are pruned since their support count or the frequency is lower than the specified minimum support value given. The elements {F, G, I, L, M} are pruned from the database and the final list contains only the following elements,

***E* = {A=5, B=6, C=5, D=5, E=6, H=3, J=3, K=4}**

Initially the itemset AB is checked for actual database and for discarded database. Note: The user defined support count is 3.

ACTUAL DATABASE

A = 1 0 0
B = 1 1 1 & AND operation
AB = 1 0 0

Here the number of 1's is counted and it is 1.

DISCARDED DATABASE

A = 0 0 0 0
B = 0 0 0 1 | OR operation
AB = 0 0 0 1

Here the number of 0's is counted and it is 3.

The total number of **{AB} = 1 + 3 = 4**, which is greater than the min_sup 3 and hence it is **frequent** and the 3-itemset is computed using AB.

ACTUAL DATABASE

AB = 1 0 0
C = 0 1 1 & AND operation
ABC = 0 0 0

Here the number of 1's is zero.

DISCARDED DATABASE

AB = 0 0 0 1
C = 1 0 0 0 | OR operation
ABC = 1 0 0 1

Here the number of 0's is counted and it is 2.

The total number of **{ABC} = 0 + 2 = 2**, which is lesser than the min_sup 3 and hence it is ***infrequent*** and all the itemsets like ABCD, ABCE, ABCH, ABCJ, ABCK are not computed and pruned.

ACTUAL DATABASE

AB = 1 0 0
D = 1 0 1 & AND operation
ABD = 1 0 0

Here the number of 1's is ONE.

DISCARDED DATABASE

AB = 0 0 0 1
D= 0 1 0 0 | OR operation
ABD = 0 1 0 1

Here the number of 0's is counted and it is 2.

The total number of **{ABD} = 1 + 2 = 3**, which is EQUAL to the min_sup 3 and hence it is ***frequent*** and retained**,** the 4-itemset is computed using {ABD}

ACTUAL DATABASE

ABD = 1 0 0
E = 1 1 1 & AND operation
ABDE = 1 0 0
Here the number of 1's is ONE.

DISCARDED DATABASE

ABD = 0 1 0 1
E = 0 0 0 0 | OR operation
ABDE = 0 1 0 1
Here the number of 0's is counted and it is 2.

The total number of **{ABDE} = 1 + 2 = 3**, which is EQUAL to the min_sup 3 and hence it is ***frequent*** and retained**,** the 5-itemset is computed using {ABDE}

ACTUAL DATABASE

ABDE = 1 0 0
H = 1 1 1 & AND operation
ABDEH = 1 0 0
Here the number of 1's is ONE.

DISCARDED DATABASE

ABDE = 0 1 0 1
H = 0 1 1 0 | OR operation
ABDEH = 0 1 1 1
Here the number of 0's is counted and it is 1.

The total number of **{ABDEH} = 1 + 1 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent.***

ACTUAL DATABASE

AB = 1 0 0
E = 1 1 0 & AND operation
ABE = 1 0 0
Here the number of 1's is ONE.

DISCARDED DATABASE

AB = 0 0 0 1

E = 0 0 0 0 | OR operation

ABE = 0 0 0 1

Here the number of 0's is counted and it is 3.

The total number of **{ABE} = 1 + 3 = 4**, which is GREATER THAN the min_sup 3 and hence it is ***frequent*** and retained**,** the 4-itemset is computed using {ABE}

ACTUAL DATABASE

ABE = 1 0 0

H = 1 0 0 & AND operation

ABEH = 1 0 0

Here the number of 1's is ONE.

DISCARDED DATABASE

ABE = 0 0 0 1

H = 0 1 1 0 | OR operation

ABEH = 0 1 1 1

Here the number of 0's is counted and it is 1.

The total number of **{ABEH} = 1 + 1 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

AB = 1 0 0

H = 1 0 0 & AND operation

ABH = 1 0 0

Here the number of 1's is ONE.

DISCARDED DATABASE

AB = 0 0 0 1

H = 0 1 1 0 | OR operation

ABH = 0 1 1 1

Here the number of 0's is counted and it is 1.

The total number of **{ABH} = 1 + 1 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

AB = 1 0 0

J = 0 0 0 & AND operation

ABJ = 0 0 0

Here the number of 1's is ZERO.

DISCARDED DATABASE

AB = 0 0 0 1
J = 0 0 1 0 | OR operation
ABJ = 0 0 1 1

Here the number of 0's is counted and it is 2.

The total number of **{ABJ} = 0 + 2 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

AB = 1 0 0
K = 0 1 0 & AND operation
ABK = 0 0 0

Here the number of 1's is ZERO.

DISCARDED DATABASE

AB = 0 0 0 1
K = 0 0 1 0 | OR operation
ABK = 0 0 1 1

Here the number of 0's is counted and it is 2.

The total number of **{ABK} = 0 + 2 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

The entire candidates related to the elements {A, B} are computed and the frequent itemsets are {AB}, {ABD}, {ABDE}, {ABE}

Similarly, the elements A and C are fetched and simple binary operations are carried out to find the frequent itemsets.

ACTUAL DATABASE

A = 1 0 0
C = 0 1 1 & AND operation
AC = 0 0 0

Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

A = 0 0 0 0
C = 1 0 0 0 | OR operation

AC = 1 0 0 0

Here the number of 0's is counted and it is 3.

The total number of **{AC} = 0 + 3 = 3**, which is EQUAL to the min_sup 3 and hence it is ***frequent*** and the 3-itemset is computed using AC.

ACTUAL DATABASE

AC = 0 0 0
D = 1 0 1 & AND operation
ACD = 0 0 0

Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

AC = 1 0 0 0
D = 0 1 0 0 | OR operation
ACD = 1 1 0 0

Here the number of 0's is counted and it is 2.

The total number of **{ACD} = 0 + 2 = 2**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

AC = 0 0 0
E = 1 1 0 & AND operation
ACE = 0 0 0

Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

AC = 1 0 0 0
E = 0 0 0 0 | OR operation
ACE = 1 0 0 0

Here the number of 0's is counted and it is 3.

The total number of **{ACE} = 0 + 3 = 3**, which is EQUAL to the min_sup 3 and hence it is ***frequent*** and the 4-itemset is computed using {ACE}.

ACTUAL DATABASE

ACE = 0 0 0
H = 1 1 0 & AND operation
ACEH = 0 0 0

Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

ACE = 1 0 0 0
H = 0 1 1 0 | OR operation
ACEH = 1 1 1 0
Here the number of 0's is counted and it is 1.

The total number of **{ACEH} = 0 + 1 = 1**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

ACE = 0 0 0
J = 0 0 0 & AND operation
ACEJ = 0 0 0
Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

ACE = 1 1 1 0
J = 0 0 1 0 | OR operation
ACEJ = 1 1 1 0
Here the number of 0's is counted and it is 1.

The total number of **{ACEJ} = 0 + 1 = 1**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

ACTUAL DATABASE

ACE = 0 0 0
K = 0 1 0 & AND operation
ACEK = 0 0 0
Here the number of 1's is counted and it is ZERO.

DISCARDED DATABASE

ACE = 1 1 1 0
K = 0 0 1 0 | OR operation
ACEK = 1 1 1 0
Here the number of 0's is counted and it is 1.

The total number of **{ACEK} = 0 + 1 = 1**, which is LESSER THAN the min_sup 3 and hence it is ***infrequent*** and PRUNED.

The entire candidates related to the elements {A, C} are computed and the frequent itemsets are {AC}, {ACE}

Similarly, for all the distinct element combinations, the computations are carried out to discover the frequent itemsets.

The pseudo code of the proposed BFPDEP algorithm is showcased below. The AND operation is carried out for the ADB elements and number of ONES are computed. The OR operation is carried out for the DDB elements and the number of ZERO are computed. The summation of the ONES and ZERO is used to check whether the itemset is frequent or infrequent based on the comparison of the summation value with the user defined minimum support value.

Algorithm BFPDEP (Input database D, Min_Sup M)

INPUT: database D, Min_Sup M
OUTPUT: Frequent Itemsets
BEGIN:

1. First load the database D
2. Scan the database D
3. Initialize RESULT [] = empty
4. PDB ← PrudentDB (Input D)
5. BinaryCONV (PDB, Distinct Elements DE)
6. Fetch the elements from actual ADB and discarded DDB
7. Compute binary AND operation on elements from ADB
8. Find the number of '1' → ONES
9. Compute binary OR operation on elements from DDB
10. Find the number of '0' → ZERO
11. Sum ← ONES + ZERO
12. If [Sum ≥ M] do begin
13. Store Itemset as frequent → RESULT
14. Proceed for the next n+1 itemset
15. Else
16. Prune the itemset
17. END if

Return RESULT
END Algorithm

The output of the BFPDEP with respect to the sample dataset is shown in the table 6.7

FREQUENT ITEMSET FOUND			
A	AB	AE	BE
B	ABE	AEH	BEH
C	ABD	AH	CD
D	ABDE	AJ	DE
E	AC	AK	DEH
J	ACE	BC	DH
K	AD	BCE	EH
L	ADE	BD	JK
M	ADEH	BDE	

Table 6.7: Frequent itemsets

CHAPTER 7

SYSTEM DESIGN

7.1 USECASE DIAGRAM

7.1.1 PREPROCESS THE DATA

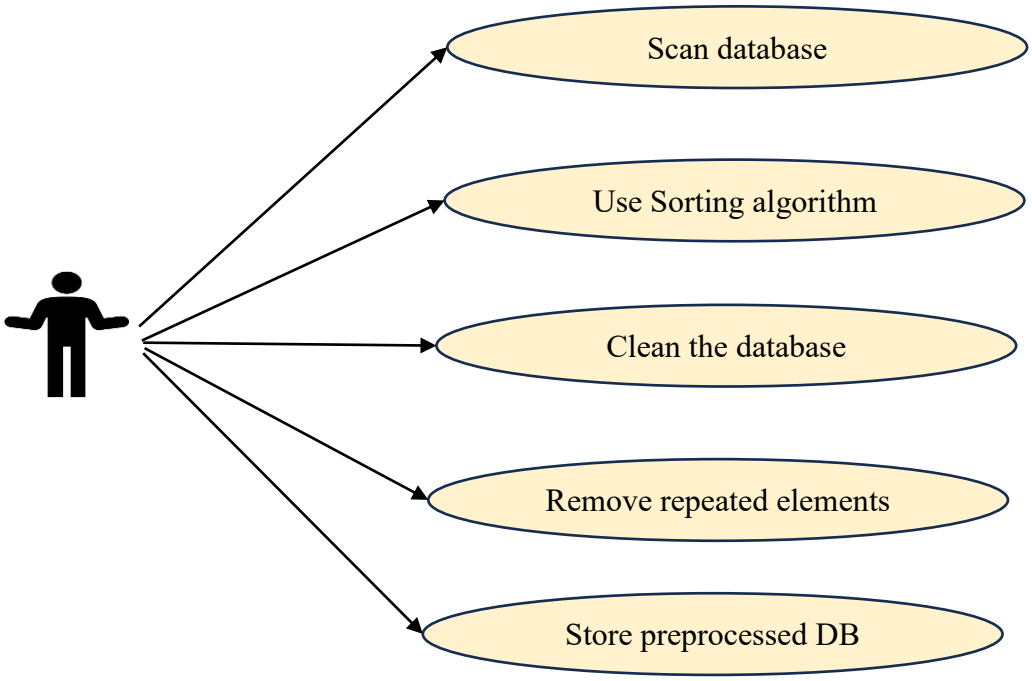


Figure 7.1 Use case diagram of the preprocessing

The above use case diagram begins with scanning the database to read its contents, followed by applying a sorting algorithm to organize the data. Next, the database is cleaned by removing irrelevant or noisy data, and duplicate elements are eliminated to reduce redundancy. Finally, the preprocessed and optimized database is stored for further analysis or mining operations.

7.1.2 FIND DISTINCT ELEMENTS

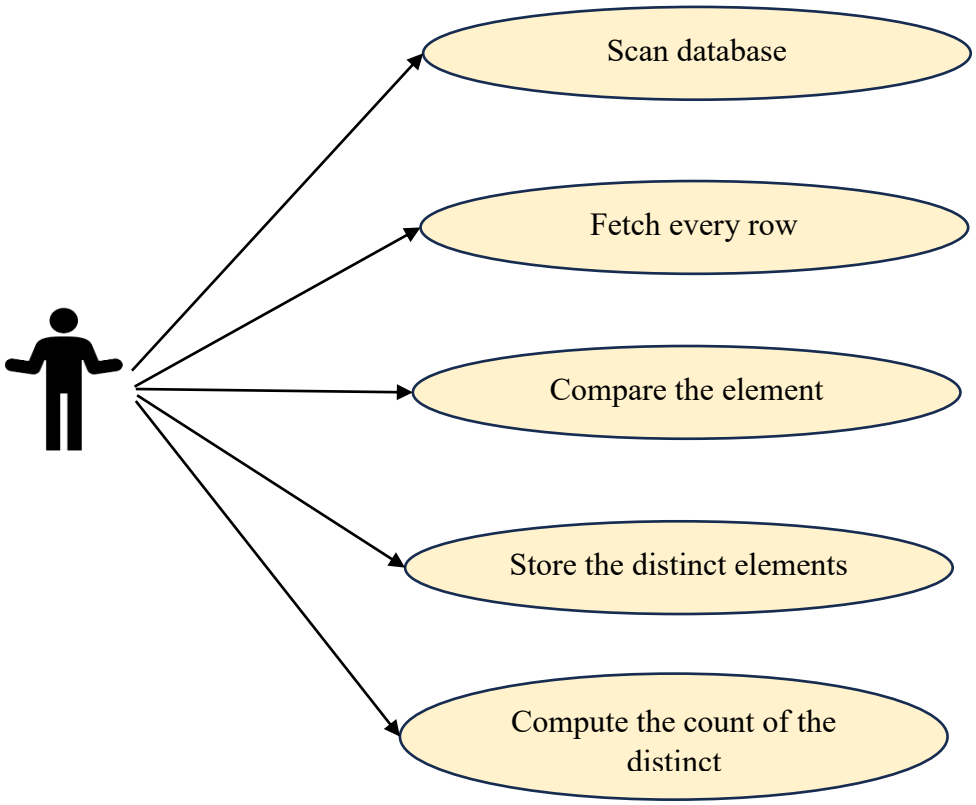


Figure 7.2 Use case diagram of the finding the distinct elements along with its count

The above use case starts with scanning the database and then fetching each row one by one. Each element is compared to check if it has already been encountered. Distinct elements are stored separately, and finally, the total count of these distinct elements is computed for further use.

7.1.3 BISECT THE DATA TO FIND THE DISCARDED

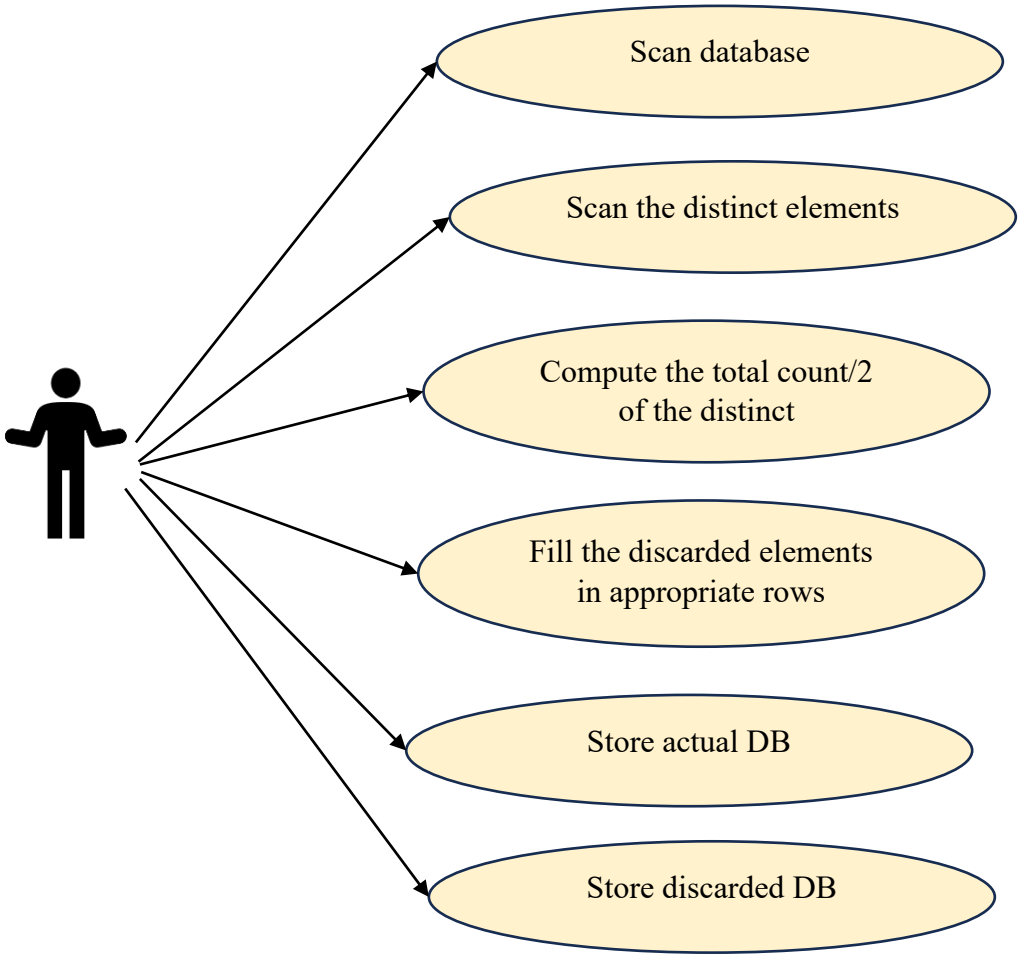


Figure 7.3 Use case diagram of the finding the store the discarded and actual database separately

The above use case starts by scanning the database and identifying the distinct elements. Then, it calculates half the total count of distinct elements as a threshold to determine which elements to discard. These discarded elements are placed into appropriate rows, and finally, the database is divided and stored as actual and discarded datasets for optimized processing or analysis.

7.1.4 BIT VECTOR REPRESENTATION

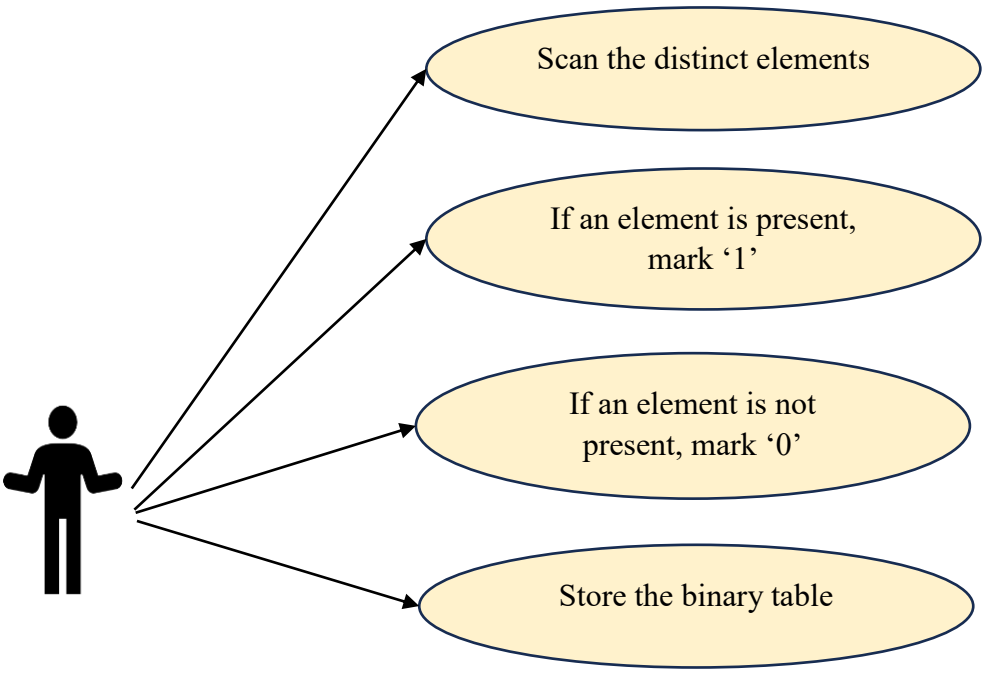


Figure 7.4 Use case diagram of the binary representation of the database

The above use case begins by scanning the distinct elements and checking each one in the dataset. If an element is present in a particular transaction or row, it is marked with a ‘1’; otherwise, it is marked with a ‘0’. The resulting binary table is then stored for further analysis, typically used in pattern mining or data compression tasks.

7.1.5 PROPOSED BFPDEP ALGORITHM

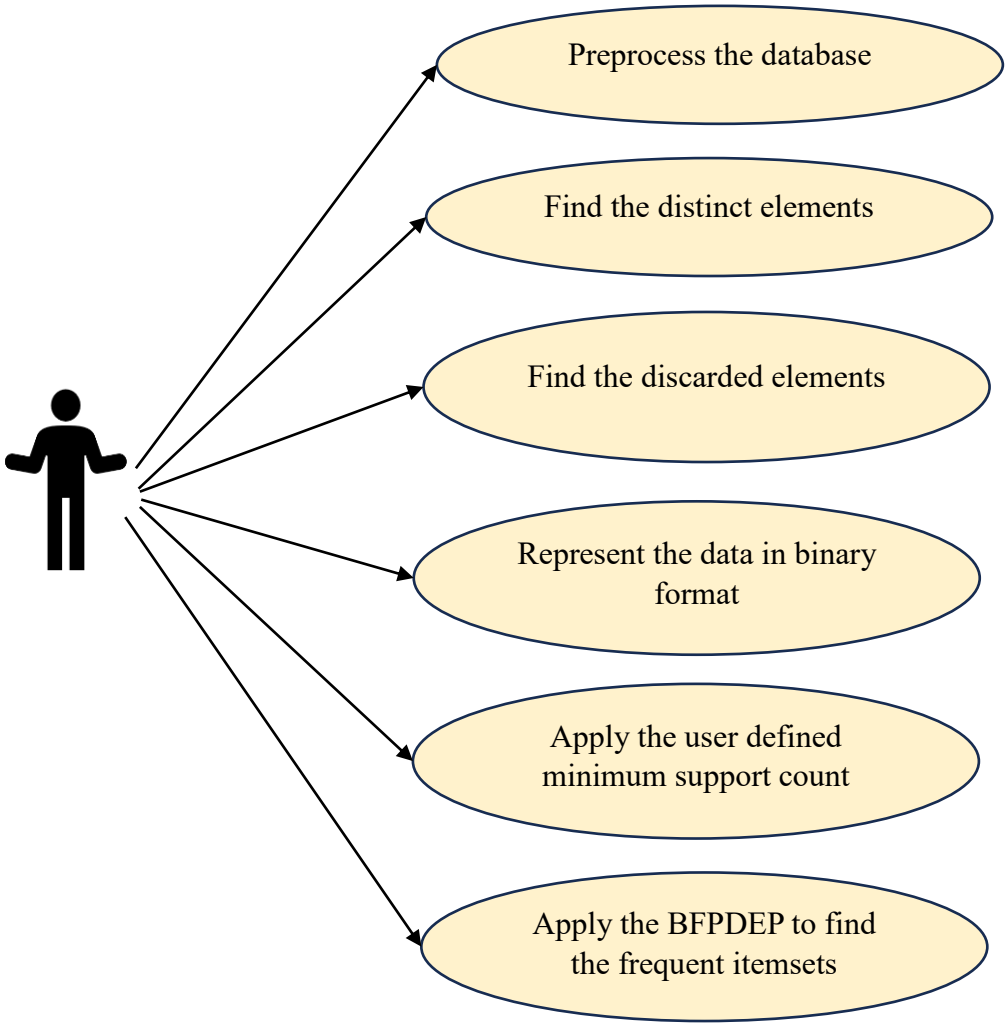


Figure 7.5 Use case diagram of the proposed BFPDEP algorithm

The above use case starts with preprocessing the database and identifying both distinct and discarded elements. The dataset is then converted into binary format for efficient computation. A user-defined minimum support count is applied to filter out infrequent patterns. Finally, the BFPDEP algorithm is used to discover the frequent itemsets from the binary representation, ensuring optimized and accurate results.

7.2 SEQUENCE DIAGRAM PREPROCESSING

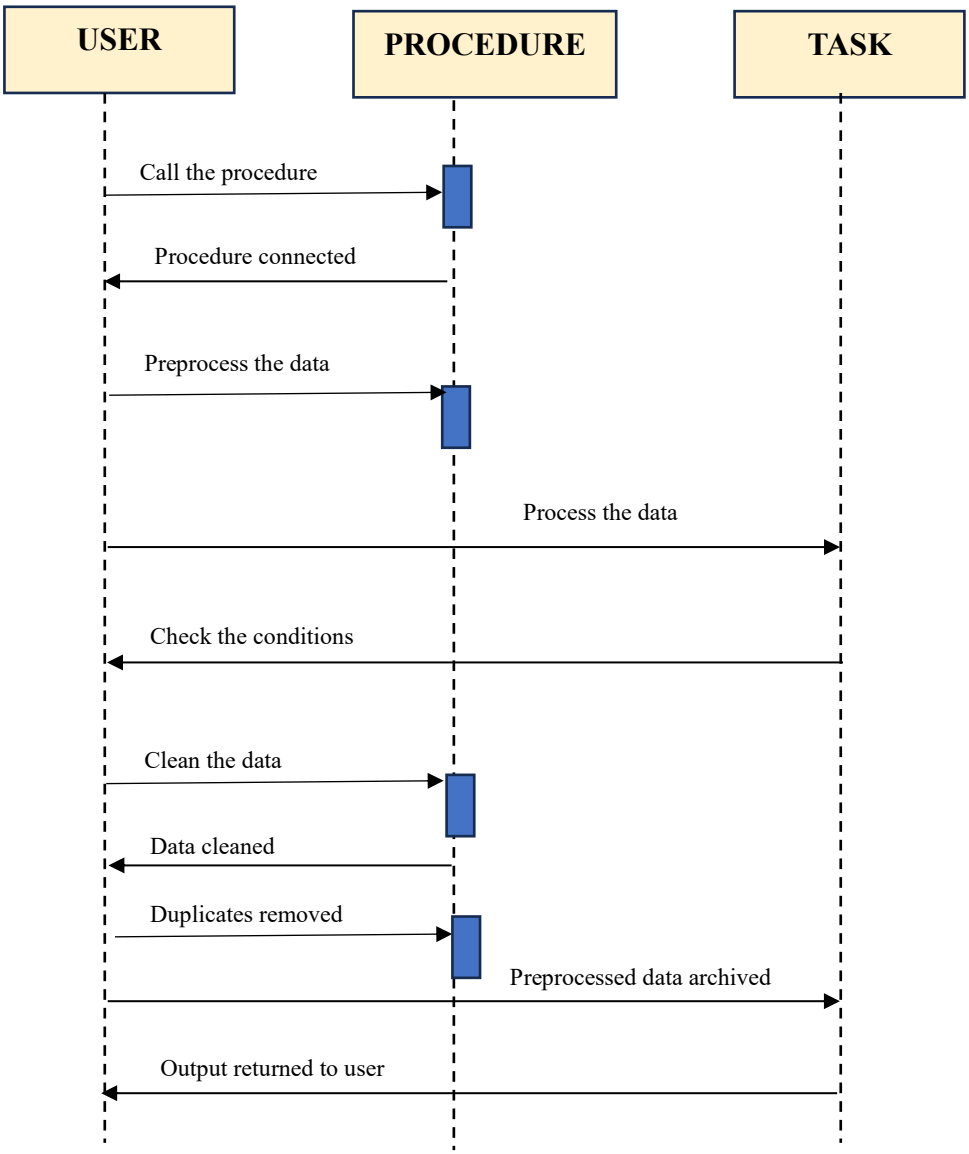


Figure 7.6 Sequence diagram of the preprocessing procedure

The process begins with the user selecting a dataset and specifying the minimum support count. The procedure component then takes over, performing data preprocessing, identifying distinct and discarded elements, and converting the data into binary format. Subsequently, the task component is responsible for computing the support counts and applying the BFPDEP algorithm to extract the frequent itemsets. This structured interaction ensures a smooth flow from user input to the generation of meaningful mining results.

7.2.1 DISTINCT ELEMENTS

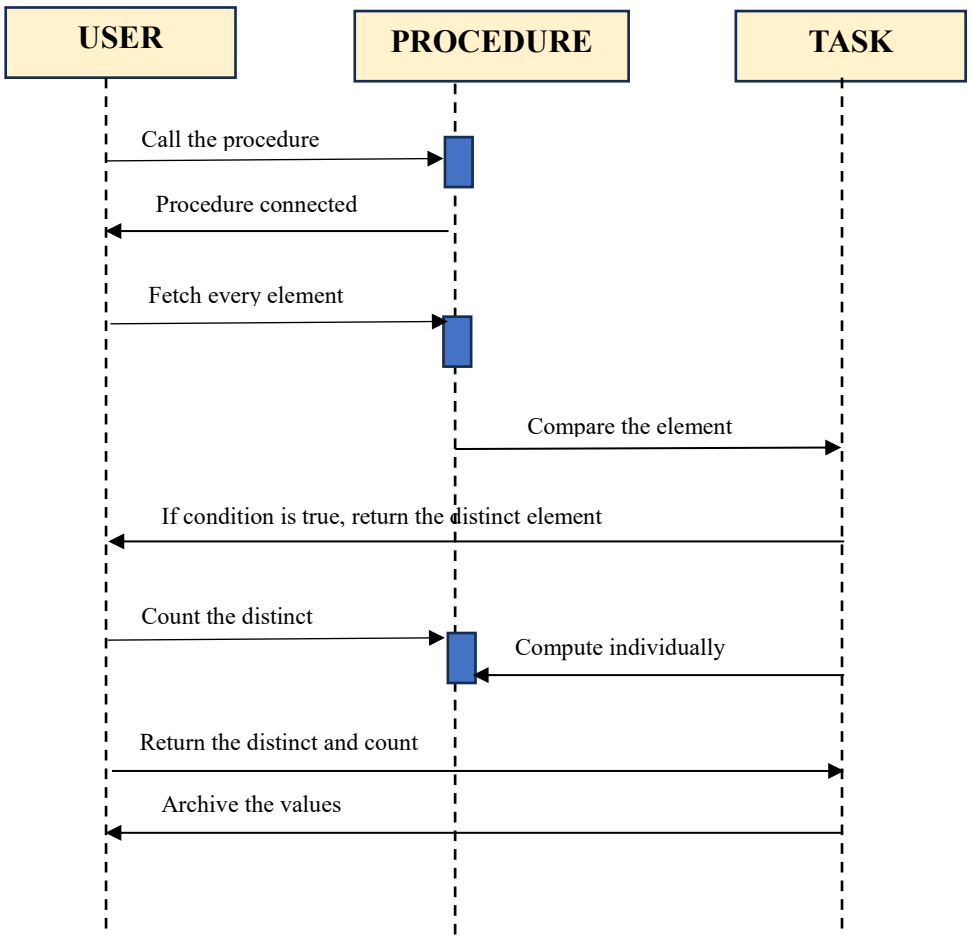


Figure 7.7 Sequence diagram to find the distinct elements procedure

The user initiates the process by selecting the database. The procedure component then scans and preprocesses the database, identifies distinct elements, and represents the data in binary format. Once the binary table is prepared, control is passed to the task component, which applies the minimum support threshold and the BFPDEP algorithm to mine the frequent itemsets. This flow ensures systematic and efficient pattern discovery from the given dataset.

7.2.2 BIT VECTOR REPRESENTATION

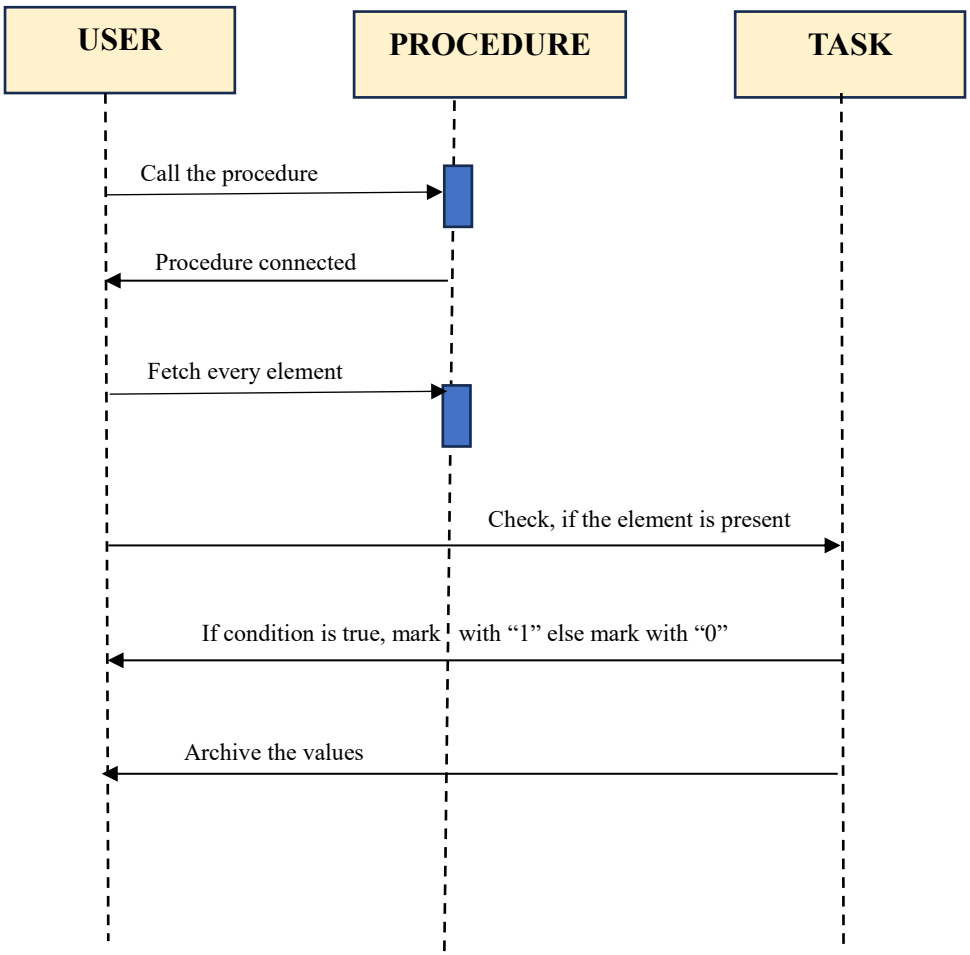


Figure 7.8 Sequence diagram to represent data in binary

The Procedure component then handles the internal operations, including scanning the dataset and converting it into binary format. Once the binary representation is ready, control is passed to the Task component, which computes the frequent itemsets based on the user-defined minimum support threshold. This clear and sequential flow ensures that the system efficiently transitions from data input to meaningful pattern extraction using the BFPDEP approach.

CHAPTER 8
TESTING AND IMPLEMENTATION

8.1 UNIT TESTING

Unit testing focuses verification effort on the smallest unit of software design, the module. The unit testing we have is white box oriented and some modules the steps are conducted in parallel.

WHITE BOX TESTING

This type of testing ensures that

- All independent paths have been exercised at least once
- All logical decisions have been exercised on their true and false sides
- All loops are executed at their boundaries and within their operational bounds
- All internal data structures have been exercised to assure their validity.

To follow the concept of white box testing we have tested each form we have created independently to verify that Data flow is correct, All conditions are exercised to check their validity, All loops are executed on their boundaries.

BASIC PATH TESTING

Established technique of flow graph with Cyclomatic complexity was used to derive test cases for all the functions. The main steps in deriving test cases were:

1. Use the design of the code and draw correspondent flow graph.
2. Determine the Cyclomatic complexity of resultant flow graph, using formula:

V(G)=E-N+2 or

V(G)=P+1 or

V(G)=Number Of Regions

Where V(G) is Cyclomatic complexity,

E is the number of edges,

N is the number of flow graph nodes,

P is the number of predicate nodes.

CONDITIONAL TESTING

In this part of the testing each of the conditions were tested to both true and false aspects. And all the resulting paths were tested. So that each path that may be generate on particular condition is traced to uncover any possible errors.

DATA FLOW TESTING

This type of testing selects the path of the program according to the location of definition and use of variables. This kind of testing was used only when some local variables were declared. The *definition-use chain* method was used in this type of testing. These were particularly useful in nested statements.

LOOP TESTING

In this type of testing all the loops are tested to all the limits possible. The following exercise was adopted for all loops:

- All the loops were tested at their limits, just above them and just below them.
- All the loops were skipped at least once.

For nested loops test the inner most loop first and then work outwards.

8.2 TESTING CONDUCTED IN THIS PROJECT

JUnit testing is essential for ensuring that your Java code works as expected. Below are the steps to get started with JUnit testing.

SETTING UP JUNIT USING MAVEN

Add the following dependencies to your pom.xml:

```
<dependencies>
  <dependency>
    <groupId>org.junit.eclipse</groupId>
```

```
        <artifactId>junit-eclipse-api</artifactId>
        <version>5.7.0</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.eclipse</groupId>
        <artifactId>junit-eclipse-engine</artifactId>
        <version>5.7.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

ASSERTIONS IN JUNIT

JUnit provides several assertion methods to verify the output of your tests:

- assertEquals(expected, actual) - Checks if the actual value is equal to the expected value.
- assertTrue(condition) - Checks if a condition is true.
- assertFalse(condition) - Checks if a condition is false.
- assertNull(object) - Checks if an object is null.
- assertNotNull(object) - Checks if an object is not null.
- assertThrows(expectedException.class, executable) - Verifies if a method throws the expected exception

8.3 MOCKING (WITH MOCKITO)

For unit tests involving dependencies, you can mock those dependencies using Mockito.

Example of mocking a repository class in a service test:

```
import static org.mockito.Mockito.*;
import org.junit.eclipse.api.Test;
```

```
import static org.junit.eclipse.api.Assertions.*;
public class ServiceTest {     @Test
   public void testServiceMethod() {
      Repository mockRepo = mock(Repository.class);
      when(mockRepo.getData()).thenReturn("Mock Data");
      Service service = new Service(mockRepo);
      String result = service.processData();
      assertEquals("Processed Mock Data", result);   }}
```

8.4 IMPLEMENTATON

A personal computer running Windows 11 with a 2.66GHz Intel I5, 1TB HDD and 16GB of RAM was used to create the proposed BFPDEP algorithm using the Java programming language. The benchmarked databases like Connect, Chess and Mushroom are sourced from the Machine Learning Database Repository at UC Irvine. Apart from these benchmarked databases, the IBM quest tool is used to create dense synthetic database to test the proposed algorithm's working.

CANDIDATE COMPARISON – CHESS					
Algorithm	**Minimum Support**				
	0.20	**0.25**	**0.30**	**0.35**	**0.50**
parEclat	73998	65339	57298	46882	22678
Bi-Eclat	53385	43880	33845	29209	17732
BigFIM	48195	37983	30898	26778	15201
BFPDEP	39006	32205	25980	20117	12400

CANDIDATE COMPARISON – MUSHROOM					
Algorithm	**Minimum Support**				
	0.10	**0.12**	**0.15**	**0.18**	**0.20**
parEclat	55628	44889	38990	32672	14982
Bi-Eclat	42661	36229	30661	25772	11798
BigFIM	43982	38665	32009	27993	13589
BFPDEP	37442	31797	28435	22235	10612

CANDIDATE COMPARISON - CONNECT4					
Algorithm	**Minimum Support**				
	0.15	**0.20**	**0.25**	**0.35**	**0.45**
parEclat	120936	112645	85745	65387	40437
Bi-Eclat	91867	77849	63523	48279	28983
BigFIM	98789	84991	72853	52687	33167
BFPDEP	83203	72894	55378	40446	25180

CANDIDATE COMPARISON - T20I16D150K					
Algorithm	**Minimum Support**				
	0.10	**0.15**	**0.20**	**0.30**	**0.45**
parEclat	546868	489457	398396	316834	166482
Bi-Eclat	418452	373883	328912	232210	113872
BigFIM	478883	383819	359073	259104	137818
BFPDEP	252295	210993	172892	157782	105488

Table 8.1: Comparison related to candidate generation

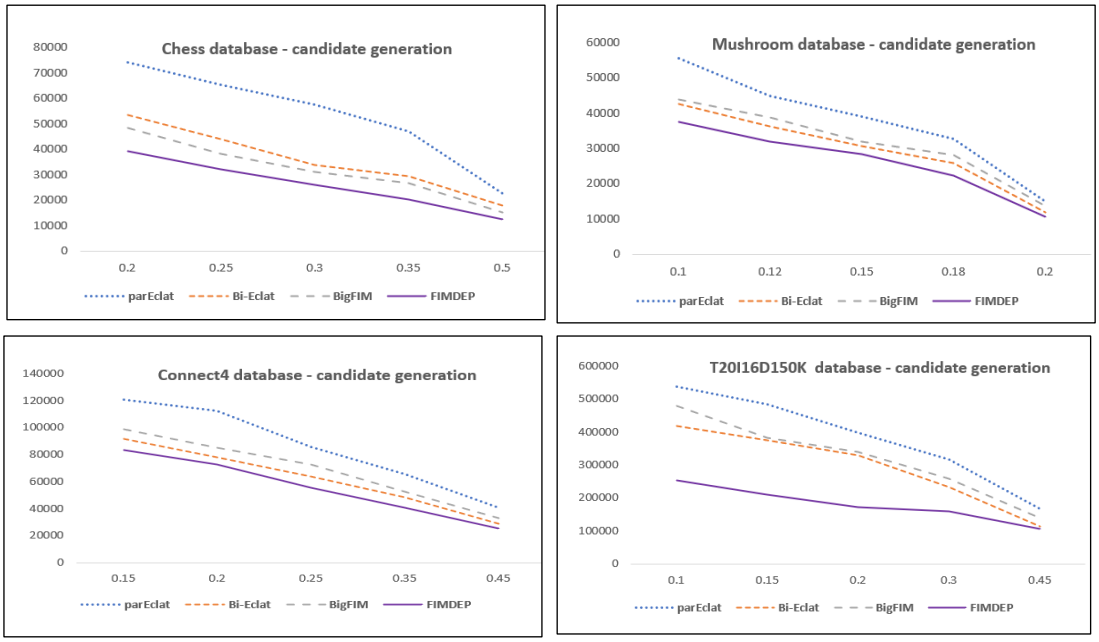


Figure 8.1: Graph representing the comparison related to candidate generation

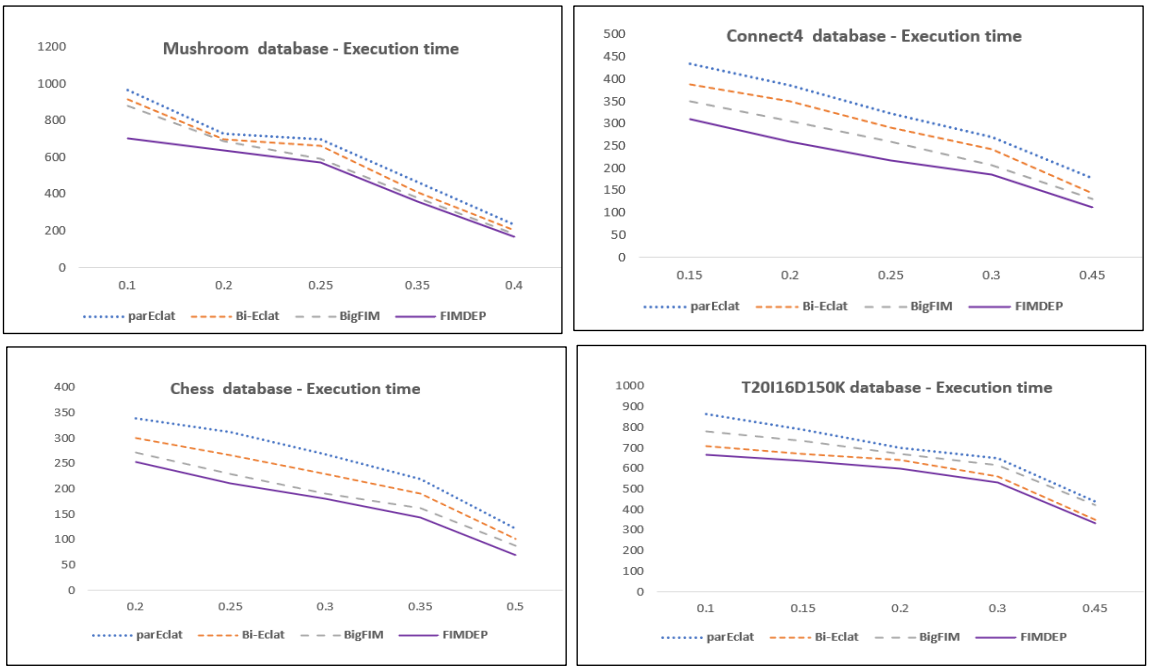


Figure 8.2: Graph representing the comparison related to execution time consumed

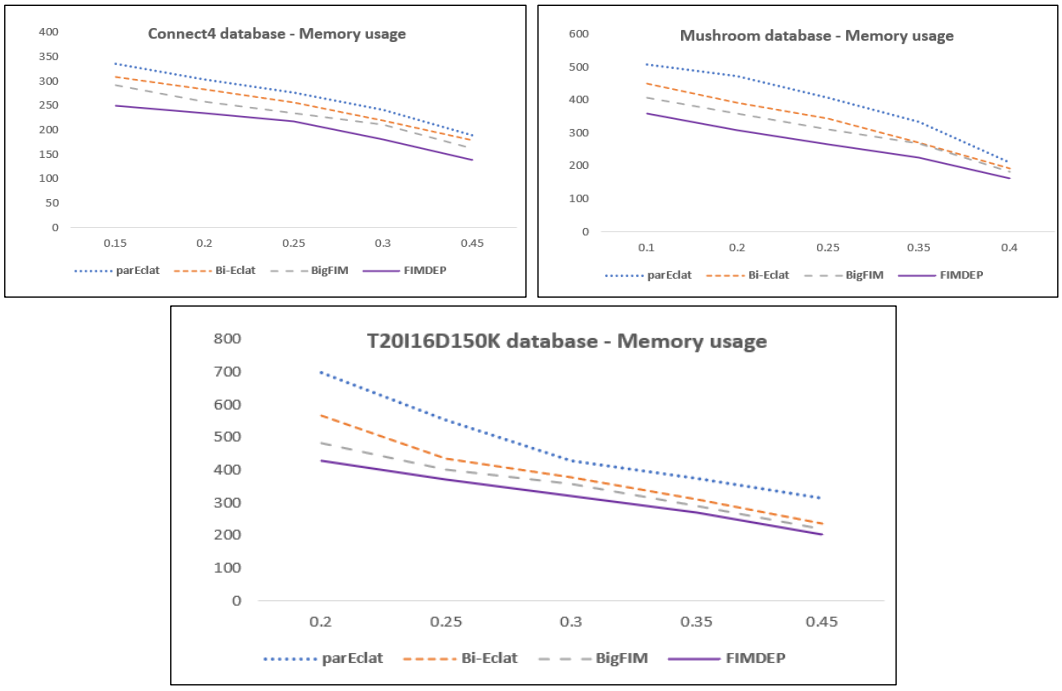


Figure 8.3: Graph representing the comparison related to memory consumption

From the above figures and tables, it is evident that the proposed algorithm BFPDEP outperformed the other three existing algorithms by a good margin when compared with few metrics like candidate generation, execution time consumption and memory consumption. When the minimum support is increased above 0.66, all the algorithms performed equally well and the proposed algorithm performed reasonably well even if the support value is decreased below 0.071.

The Bi-Eclat performed reasonably well on Connect, Mushroom and synthetic databases and it was the best performing algorithm in the existing algorithm list. The BigFIM performed quite well on the Chess database like the proposed BFPDEP. The main reason behind the performance of the proposed algorithm lies in its superior data representation and pruning mechanism employed. The databases Connect4, mushroom and chess saw a reduction of 21%, 26% and 32% in its elements after the proposed algorithm BFPDEP inducted the discarded element representation,

Initially, in the proposed algorithm BFPDEP the discarded arrangement makes it easier and faster to compute itemset support since it only uses the presence of 1’s and 0’s (compressed bit-vectors format stored as binary digits). Second, the vertical arrangement automatically "reduces" the database size before each scan by accessing only those itemsets from disk that are pertinent to the subsequent mining process scan.

CHAPTER 9
CONCLUSION

The study was meticulously designed and executed with the objective of addressing the critical challenges in the field of frequent itemset mining. Recognizing the growing demand for efficient algorithms capable of handling large-scale data with minimal computational overhead, the proposed BFPDEP (Bit vector-based Frequent Pattern mining using Discarded Elements Prudently) algorithm presents a tailored solution to the major limitations present in conventional techniques—particularly those related to time complexity and memory usage.

The research involved both real-time and synthetic databases to ensure the robustness and general applicability of the approach. A comprehensive evaluation was performed, focusing on key performance metrics such as execution time, the number of candidate itemsets generated, and overall memory consumption. These aspects are crucial for determining the practicality of an algorithm in real-world scenarios, especially where large volumes of transactional data are involved.

The comparative analysis clearly demonstrated that the BFPDEP algorithm outperforms existing methods. By effectively utilizing bit vector representations and prudently managing discarded elements, the algorithm significantly reduces redundancy in computation and storage. The experimental results validated that BFPDEP not only minimized the number of unnecessary candidate itemsets but also achieved faster execution with lower memory usage.

In conclusion, the BFPDEP algorithm offers a substantial advancement in the domain of frequent itemset mining. It successfully bridges the gap between performance and scalability, making it a promising solution for data mining tasks in environments where efficiency and resource management are of paramount importance. Future work may further optimize this method or extend it to other data mining problems, but the current outcomes firmly establish BFPDEP as a highly effective and innovative approach in this space.

CHAPTER 10
FUTURE ENHANCEMENT

10.1 FUTURE RECOMMENDATIONS

With the help of the divide and conquer strategy and compression technique, the proposed algorithms seem to be effective in terms of run time and memory use; yet, there is undoubtedly room for more study and development.

The pruning processes may be improved to ensure that the fewest possible itemset is generated with minimal execution time and memory footprints, and more crucially, with high execution speed in generating significant result patterns.

The proposed technique may be slightly modified to meet the needs of the big data, and data compression can be used to lessen the strain of the large memory footprints while dealing with big data.

Given the daily, massive, and fast growth in data size, a hybrid technique that combines the best features of existing algorithms can be used to develop a new algorithm that further improves performance. By integrating the capabilities of some good existing algorithms, the BFPDEP method may be used to identify sequential patterns in online data that is widely dispersed around the world.

APPENDIX I

SAMPLE CODING

FrequentItemset.java

```
package FinalYearProject2;
import java.util.ArrayList;
import java.util.Map;
import java.util.HashMap;
public class FrequentItemset {
        static int minimum_support;
        static int q;
   public static Map<String,ArrayList<?>> frequentItemSet(
      ArrayList<String> uniqueList,
      ArrayList<Integer> c,
      ArrayList<Integer> c1,
      Map<String,ArrayList<Integer>> actualBvr,
      Map<String,ArrayList<Integer>> discardBvr,
      int minimum_support) {
      ArrayList<ArrayList<String>> frequent = new ArrayList<>();
      ArrayList<String> oneItemSet = new ArrayList<>();
      ArrayList<Integer> support = new ArrayList<Integer>();
      Map<String,ArrayList<?>> fin= new HashMap<>();
      for (int i=0;i<c.size();i++) {
         int k=c.get(i)+c1.get(i);
         if (k>=minimum_support) {
            oneItemSet.add(uniqueList.get(i));
         }
      }
      generateFrequentItemsets(oneItemSet,new
ArrayList<>(),0,frequent,actualBvr,discardBvr,minimum_support,support);
```

```
        fin.put("support", support);
        fin.put("frequent",frequent);
        return fin;
    }
    private static void generateFrequentItemsets(
        ArrayList<String> oneItemSet,
        ArrayList<String> currentSubset,
        int start,
        ArrayList<ArrayList<String>> frequent,
        Map<String,ArrayList<Integer>> actualBvr,
        Map<String,ArrayList<Integer>> discardBvr,
        int minimum_support,
        ArrayList<Integer> support) {

        if (!currentSubset.isEmpty()) {
            ArrayList<Integer> actual_subset = bitOperation(currentSubset, actualBvr);
            ArrayList<Integer> discard_subset = bitOperation(currentSubset,
discardBvr);

            int a_ct=0;
            for (int value : actual_subset) {
                if (value==1) {
                    a_ct++;
                }
            }

            int d_ct=0;
            for (int value : discard_subset) {
                if (value==1) {
```

```
                d_ct++;
            }
        }
        q=a_ct+d_ct;
        if (q>=minimum_support && currentSubset.size()>1) {
            frequent.add(new ArrayList<>(currentSubset));
            support.add(q);
        }
    }
    for (int i=start;i<oneItemSet.size();i++) {
        currentSubset.add(oneItemSet.get(i));
        generateFrequentItemsets(oneItemSet,currentSubset,i +
1,frequent,actualBvr,discardBvr,minimum_support,support);
        currentSubset.remove(currentSubset.size()-1);
    }
  }

        public static ArrayList<Integer> bitOperation(
                        ArrayList<String> temp,
                        Map<String,ArrayList<Integer>> bvr) {
                ArrayList<ArrayList<Integer>> temp_bit=new
ArrayList<ArrayList<Integer>>();
                ArrayList<Integer> sub_res=new ArrayList<Integer>();
                for(int i=0;i<temp.size();i++) {
                        temp_bit.add(bvr.get(temp.get(i)));
                }
                int k=0,b,size;
                size=temp_bit.get(0).size();
                while(k!=size) {
```

```
                b=1;
            for(int j=0;j<temp_bit.size();j++) {

                b=b & temp_bit.get(j).get(k);
            }
            sub_res.add(b);
            k++;
            }
            return sub_res;

        }
}
```

BVR.java

```
package FinalYearProject2;
import java.util.ArrayList;
import java.util.Map;
import java.util.LinkedHashMap;
public class BVR {
        public static Map<String, ArrayList<Integer>> bitVector(
                ArrayList<String> uniqueList,
                ArrayList<String> updated){
            int i,j,k=0;
            String[] a;
            ArrayList<ArrayList<Integer>> bvr=new
ArrayList<ArrayList<Integer>>();
            ArrayList<Integer> inbvr;
            ArrayList<String> temp;
            for(k=0;k<uniqueList.size();k++) {
                inbvr=new ArrayList<Integer>();
```

```
                    for(i=0;i<updated.size();i++) {
                        temp=new ArrayList<String>();
                      a=updated.get(i).split(",");
                        for(j=0;j<a.length;j++) {
                            temp.add((a[j]));
                        }
                        if(temp.contains(uniqueList.get(k))) {
                            inbvr.add(1);
                        }
                        else {
                            inbvr.add(0);
                        }

                    }
                    bvr.add(inbvr);
                }
                  Map<String, ArrayList<Integer>> bvrMap = new
LinkedHashMap<>();
            for (int index = 0; index < uniqueList.size(); index++) {
               bvrMap.put(uniqueList.get(index), bvr.get(index));
            }
            return bvrMap;

        }
}
```

UniqueItems.java

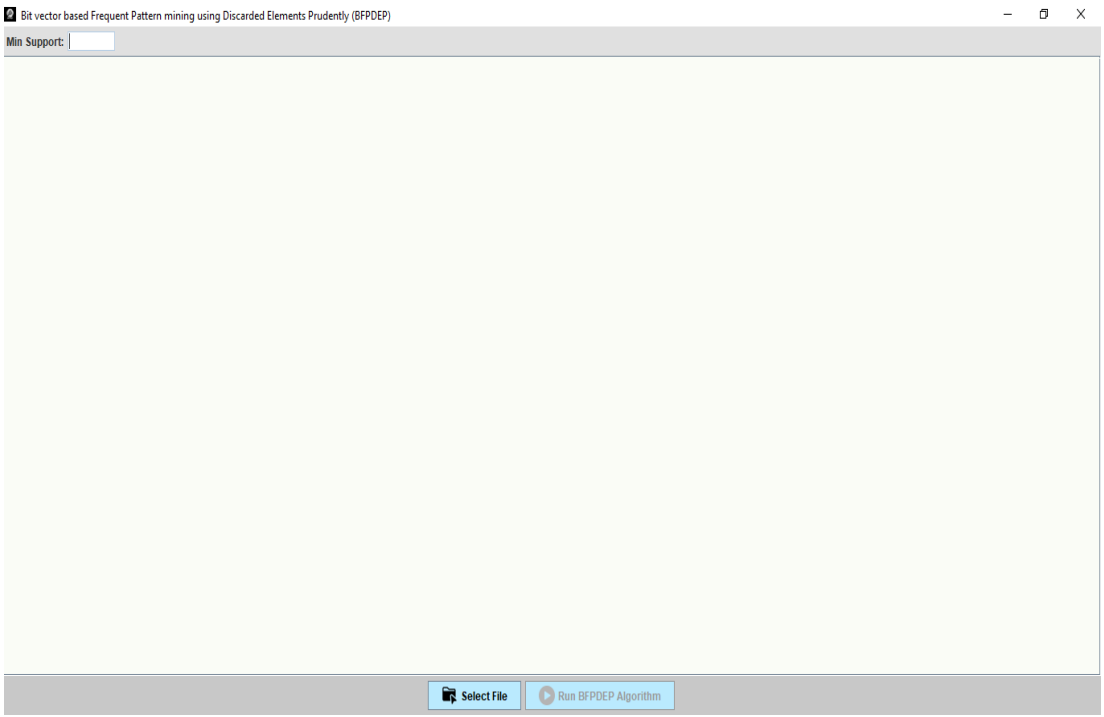
```
package FinalYearProject2;
import java.util.*;
public class UniqueItems {
```

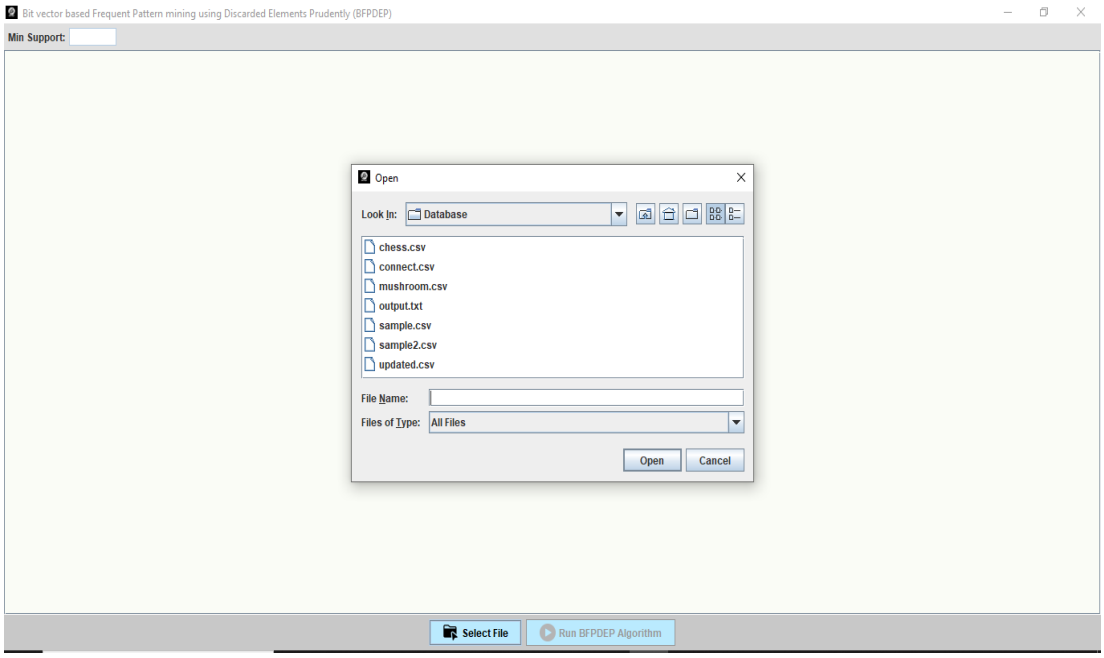
```
    public static ArrayList<String> uniqueItem(ArrayList<String> data) {
        String[] a;
        HashSet<String>unique=new HashSet<String>();
        for(int i=0;i<data.size();i++) {
            a=data.get(i).split(",");
            for(int j=0;j<a.length;j++) {
                String n=a[j];
                unique.add(n);
            }
        }
        ArrayList<String> uniqueList=new ArrayList<String>(unique);
        Collections.sort(uniqueList);
        return uniqueList;
    }

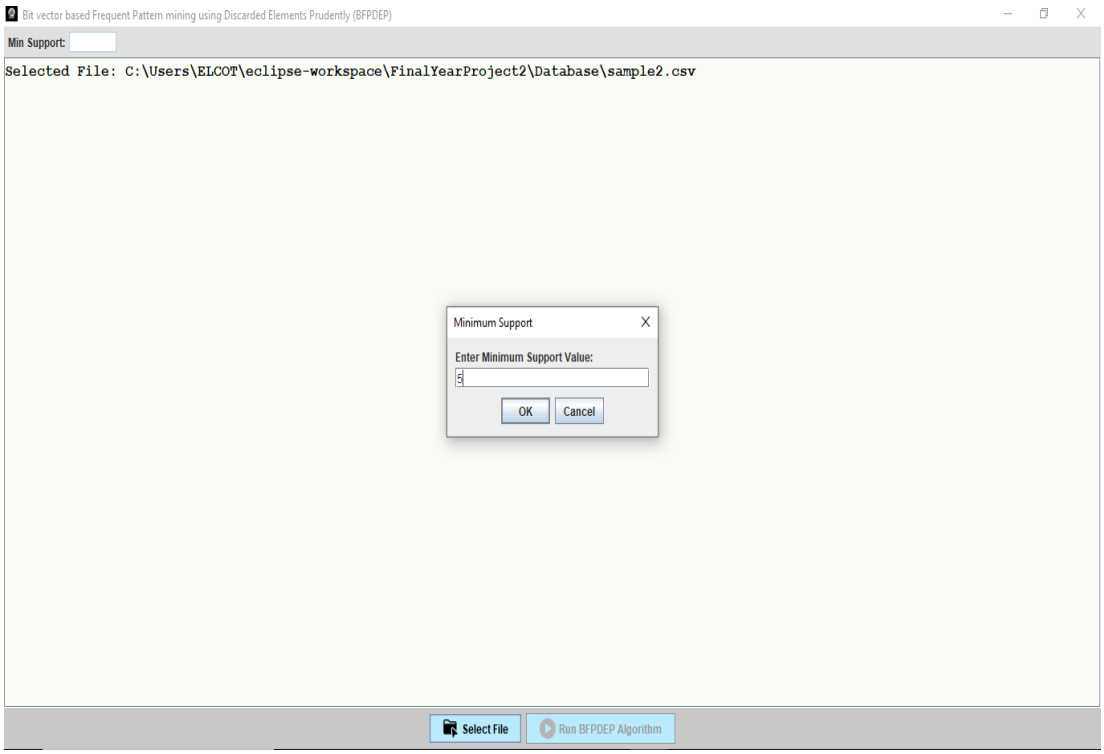
}
```

APPENDIX II

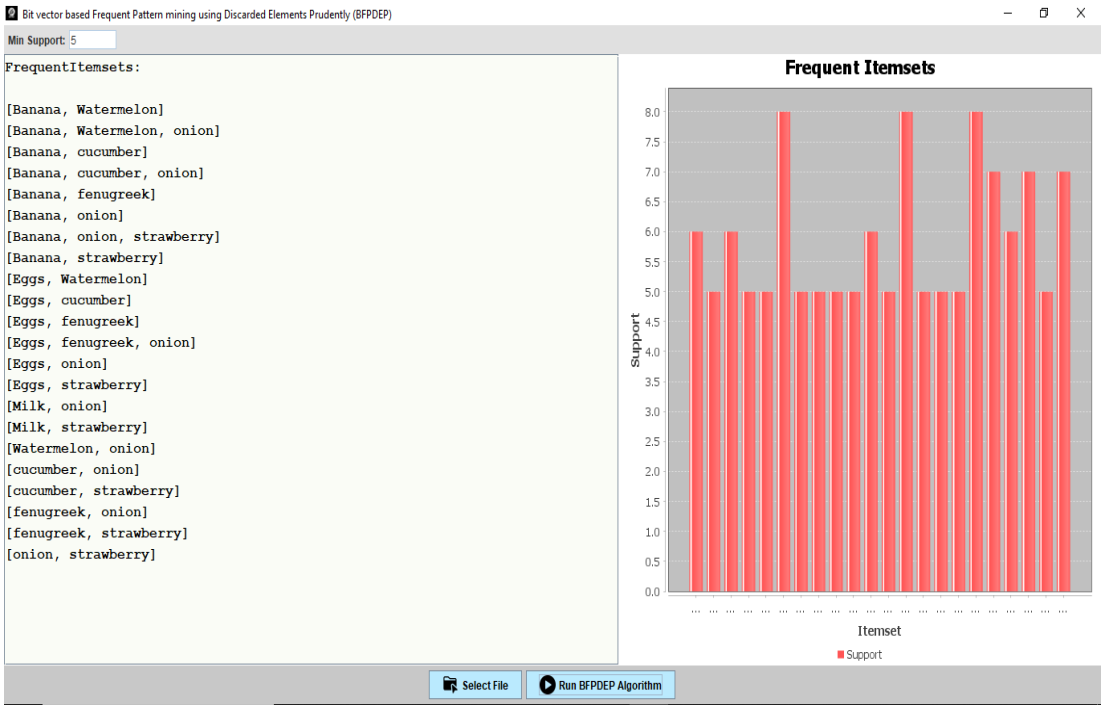
SCREENSHOTS

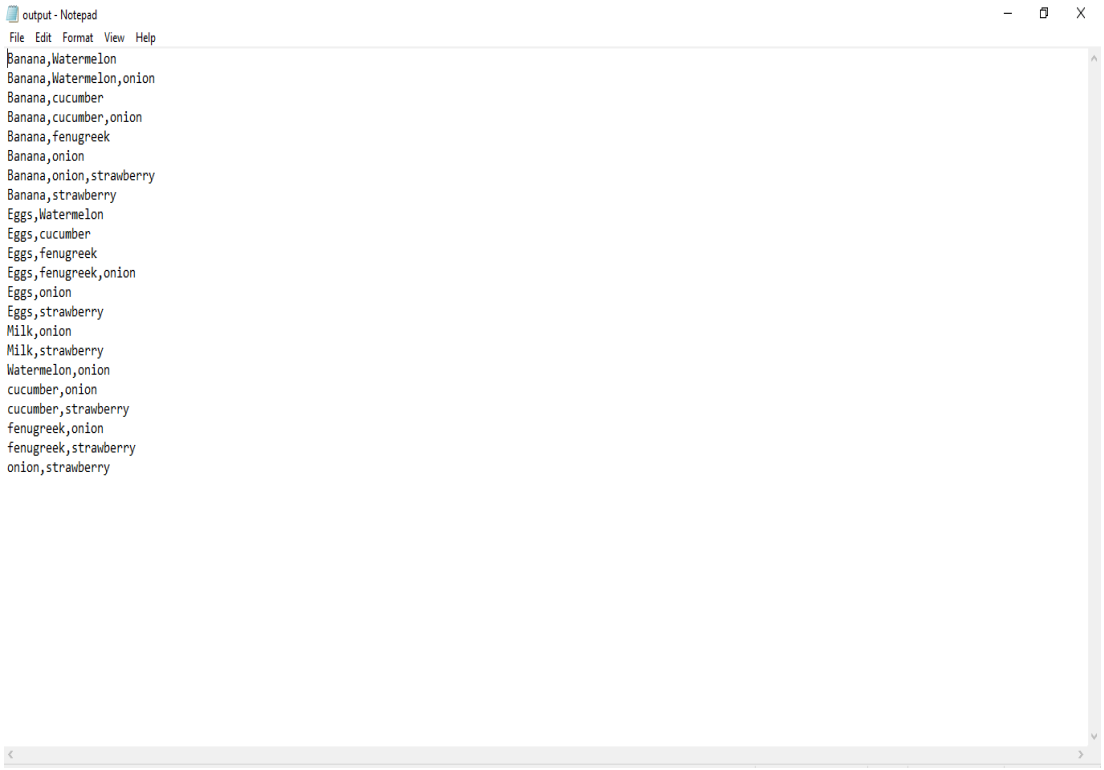





Bit vector based Frequent Pattern mining using Discarded Elements Prudently (BFPDEP)
Min Support:
Selected File: C:\Users\ELCOT\eclipse-workspace\FinalYearProject2\Database\sample2.csv
Minimum Support
Enter Minimum Support Value:
5
OK
Cancel
Select File
Run BFPDEP Algorithm


Bit vector based Frequent Pattern mining using Discarded Elements Prudently (BFPDEP)
Min Support: 5
Selected File: C:\Users\ELCOT\eclipse-workspace\FinalYearProject2\Database\sample2.csv
Select File
Run BFPDEP Algorithm


Bit vector based Frequent Pattern mining using Discarded Elements Prudently (BFPDEP)
Min Support: 5
FrequentItemsets:
[Banana, Watermelon]
[Banana, Watermelon, onion]
[Banana, cucumber]
[Banana, cucumber, onion]
[Banana, fenugreek]
[Banana, onion]
[Banana, onion, strawberry]
[Banana, strawberry]
[Eggs, Watermelon]
[Eggs, cucumber]
[Eggs, fenugreek]
[Eggs, fenugreek, onion]
[Eggs, onion]
[Eggs, strawberry]
[Milk, onion]
[Milk, strawberry]
[Watermelon, onion]
[cucumber, onion]
[cucumber, strawberry]
[fenugreek, onion]
[fenugreek, strawberry]
[onion, strawberry]
Frequent Itemsets
Support
Itemset
Support
Select File
Run BFPDEP Algorithm


output - Notepad
File Edit Format View Help
Banana,Watermelon
Banana,Watermelon,onion
Banana,cucumber
Banana,cucumber,onion
Banana,fenugreek
Banana,onion
Banana,onion,strawberry
Banana,strawberry
Eggs,Watermelon
Eggs,cucumber
Eggs,fenugreek
Eggs,fenugreek,onion
Eggs,onion
Eggs,strawberry
Milk,onion
Milk,strawberry
Watermelon,onion
cucumber,onion
cucumber,strawberry
fenugreek,onion
fenugreek,strawberry
onion,strawberry

REFERENCES

[1] Alnazzawi.N, Rauf.A, Rehman.S.U and Sadeepqullah.M, “Probabilistic Support Prediction: Fast Frequent Itemset Mining n Dense Data,” in IEEE Access, vol. 12, pp. 39330-39350, 2024, doi: 10.1109/ACCESS.2024.3376477.

[2] Ahmed.U, Djenouri.Y, Hong.T.-P, Lin.J. C.-W, Srivastava.0-G and Wu J. M.-T, “Linguistic frequent pattern mining using a compressed structure,” Applied Intelligence, vol. 51, no. 7, pp. 4806–4823, Jan. 2021.

[3] Sharmila.S and Vijayarani.S, “Association rule mining using fuzzy logic and whale optimization algorithm,” Soft Computing, vol. 25, no. 2, pp. 1431–1446, Aug. 2020.

[4] Ganesan.V, RajaRajeswari.P and Waghere.S.S, “Retrieval of frequent itemset using improved mining algorithm in hadoop,” in Advances in Intelligent Systems and Computing. Springer Singapore, Jul. 2020, pp. 787–798.

[5] Fernandez.C.G, Gastaldi.L and Lessanibahri.S, “A novel pruning algorithm ´ for mining long and maximum length frequent itemsets,” Expert Systems with Applications, vol. 142, p. 113004, Mar. 2020

[6] Patil.N and Vanahalli.M.K,“An efficient dynamic switching algorithm for mining colossal closed itemsets from high dimensional datasets,” Data and Knowledge Engineering, vol. 123, p. 101721, Sep. 2019.

[7] Bande-Serrano.J.M, Bustio-Mart´ınez.L, Cumplido.R, Feregrino-´ Uribe.C, Hernandez-Le´on.R and Letras-Luna.M,“Using hashing and lexicographic order for frequent itemsets mining on data streams,” Journal of Parallel and Distributed Computing, vol. 125, pp. 58–71, Mar. 2019.

[8] Jiang.Z.L, Liao.Q, Tian.P, Wang.X, Zhang.C and Zhang.X,“HashEclat: an efficient frequent itemset algorithm,” International Journal of Machine Learning and Cybernetics, vol. 10, no. 11, pp. 3003–3016, Jan. 2019

[9] Kameya.Y, “An exhaustive covering approach to parameter-free mining of nonredundant discriminative itemsets,” in International Conference on Big Data Analytics and Knowledge Discovery. Springer, 2016, pp. 143–159.

[10] Hadzic.F and Shaharanee.I.N.M, “Evaluation and optimization of frequent, closed and maximal association rule-based classification,” Statistics and Computing, vol. 24, no. 5, pp. 821–843, Jun. 2013.

[11] CHEN.H, HUANG.C -Z and LI.H.-F,“An approximate non-derivable itemset mining algorithm over data streams,” Chinese Journal of Computers, vol. 33, no. 8, pp. 1427–1436, Dec. 2010

[12] Georgiopoulos.M, Koufakou.A and Secretan.J, “Non-derivable itemsets for fast outlier detection in large high-dimensional categorical data,” Knowledge and Information Systems, vol. 29, no. 3, pp. 697–725, Dec. 2010.

[13] Calders.T and Goethals.B, “Non-derivable itemset mining,” Data Mining and Knowledge Discovery, vol. 14, no. 1, pp. 171–206, Jan. 2007. [Online]. Available: https://doi.org/10.1007/s10618-006-0054-6

[14] Bole.S, Bing.L, Shouzhi.Z, Wei.W, Xiaoming.Y and Zhibin.W, “Non-almost derivable frequent itemsets mining,” in The Fifth International Conference on Computer and Information Technology (CIT’05). IEEE, 2005, pp. 157–161.

[15] Calders.T and Goethals.B, “Depth-first non-derivable itemset mining,” in Proceedings of the 2005 SIAM International Conference on Data Mining. Society for Industrial and Applied Mathematics, Jan. 2005.

[16] Mitsunori Ogihara, Mohammed J. Zaki, Srinivasan Parthasarathy and Wei Li. Parallel algorithms for discovery of association rules. Data Mining and Knowledge Discovery, 1(4):343{373, 1997.

[17] Agarwal.R and Srikant.R. Mining Sequential Pattern. In Proc. 1995 Int. Conf. Data Engineering, pages 3-10.